FlintNDE：矩阵微分方程的高精度局部级数输运

2026-10-27 草稿

摘要

本文介绍 `FlintNDE`：一个建立在 FLINT 之上、对一阶矩阵微分方程做任意精度数值延拓的 Python 程序包。它的第一个核心函数沿用户给定路径用局部级数输运边界向量，自动区分普通点与正则奇点，做精确优先的 Frobenius 与对数分析、精确的 Lee–Moser 投影平衡、经过认证的指数扇区，并配一条独立的精化链。第二个核心函数在若干固定调节参数值处重复同一求解器，重建一个半解析 Laurent 级数：幂次是解析信息，系数向量是高精度数值。这样就不必让符号化的调节参数展开穿过每一个中转点。精确 Gaussian 有理矩阵给出自动的有限奇点与无穷远奇点清单；一般解析求值器用 Cauchy–DFT 提取系数，不做符号求导。高阶极点起初只证明给定基是非 Fuchs 型。随后 FlintNDE 应用由前两个 Laurent 矩阵构造的精确投影平衡、严格解耦的指数乘幂乘对数拟设；对于秩一不规则点且前导根互异的简单情形，还提供只在起点使用的形式指数级数，并给出固定五阶的诊断量。分支型或代数型的形式约化、Stokes 矩阵以及一般的 Levelt–Turrittin 问题保持拒绝（fail closed）。

1 引言

1.1 平直时空的费曼积分

微扰量子场论的振幅由一族族圈积分表达。分部积分（IBP）恒等式把一族积分约化到有限个主积分 [1]；再对主积分关于运动学、质量或辅助参数求导并重新约化，就得到一个闭合的一阶矩阵微分方程 [2]。剩下的数值问题是给定边界数据、跨越奇点区域，并在物理点上求值。

已有的公开工具以互补的方式实现了这条 IBP–DE 流程。`DiffExp` 沿相空间中的线段拼接一维局部级数 [3]。AMFlow 把 IBP 约化、辅助质量的微分方程、系统性的大质量边界数据和高精度数值输运组合起来，计算一般的维数正规化圈积分 [4, 5]。FlintNDE 采用 AMFlow 的一个具体部件：在若干固定调节参数值处求解完整问题，再数值重建 Laurent 系数。当路径很长、中转点很多时，这样做可以避免让解析的调节参数信息逐级穿过每一次中间矩阵乘法——那样做会令代数复杂度和内存占用迅速膨胀。重建仍然只是数值的：它由独立的调节参数采样检验，不作为精确的符号证明。

AmpRed 在费曼参数空间给出另一条通用的 IBP–DE 路线 [6]。它把参数积分组织成 Lee–Pomeransky（LP）表示 [7]，用参数 IBP 关系构造主积分系统，用区域展开 [8, 9] 组织渐近边界数据。FlintNDE 不替代任何约化或区域分析前端。它接收这些前端产出的单变量矩阵系统和已认证的边界数据。

1.2 de Sitter 与宇宙学积分

暴胀期间的大质量场会留下特征性的非高斯信号，这推动了准单场和宇宙学对撞机等研究方向 [10, 11, 12]。这里只提及直接构造 IBP 关系与微分方程的技术。dSIBP 系列的前两篇文章给出时间积分的一般化 IBP 关系并求解相应的系统，其中包括多元超几何与 $\mathrm{d}\log$ 形式 [13, 14]。其圈图推广把时间积分和圈动量积分放进同一个按宇称分块的 IBP/DE 系统 [15]。

另一条互补的能量空间路线把时间积分换成能量变量，并通过运动学流导出微分方程 [16, 17]。同类能量表示中的上同调 IBP 及其大质量关联函数的推广给出相关的实现 [18, 19]。这些应用说明需要一个求解器，它不关心矩阵来自平直时空圈图族、宇宙学时间积分，还是时间–圈动量联合族。

1.3 共同的数值问题与程序包的定位

尽管积分表示各不相同，上面这些平直时空和宇宙学的系统最终归结到同一个下游数值任务：把

$$\frac{\mathrm{d}Y(s)}{\mathrm{d}s} = A(s,\epsilon)Y(s) \tag{1}$$

的向量解从已认证的边界数据延拓到目标点，同时在奇点处保持局部解空间不变，并控制沿路径的截断误差和工作精度误差。延拓链很长时，这个区分很关键：普通 Taylor 输运、Frobenius 幂–对数输运和调节参数重建解决的是不同的问题，不能互相替代。

FlintNDE 从约化之后开始。它的输入约定是一个单变量一阶矩阵 DE、边界数据和一条直接给出的点列表；它不生成 IBP 恒等式，也不假定某种特定的主积分记号。对精确的 Gaussian 有理输入，它判定有限奇点和无穷远奇点，并做精确优先的指标、Jordan 和共振检验；对一般解析输入，它用 Cauchy–DFT 采样数值计算普通点的 Taylor 系数。两条路线都用 FLINT 的任意精度复球算术做实际输运，并都用一条独立的精化链控制误差。

程序包提供两条工作流。第一条在普通点和正则奇点之间输运解向量；只要局部结构需要对数，就使用统一的幂–对数递推。第二条在固定调节参数值处反复调用同一输运，重建 Laurent 级数的数值系数。极点阶数大于一时，首先报告的是“给定基为非 Fuchs 型”。随后局部调度器施加精确的 Lee–Moser 投影平衡；若仍有高阶极点，则或者给出彼此解耦的标量指数扇区，或者给出 Sec. 6.4 描述的受限秩一形式递推。后者是初值边界求值器，不是 Stokes 连接求解器。分支型、代数型以及一般的 Levelt–Turrittin/Stokes 问题不在当前实现范围内。接下来两节给出完整流程；安装、例子以及数学和软件细节放在后面。

2 核心功能一：数值矩阵 DE 输运

基本函数用 FLINT 任意精度复球算术，让一个 N 分量的边界向量穿过一条直接给出的点列表。它接受精确的 $\mathbb{Q}(i)(s)$ 矩阵，也接受一般解析的矩阵求值器。凡是会改变解空间的精确结构——奇点、指标根、Jordan 块和共振——都用精确数据判定；普通点的系数和输运则是数值的。

图 1 给出完整的基本流程。对数分析在这里只作为一个节点出现，其展开见图 3。

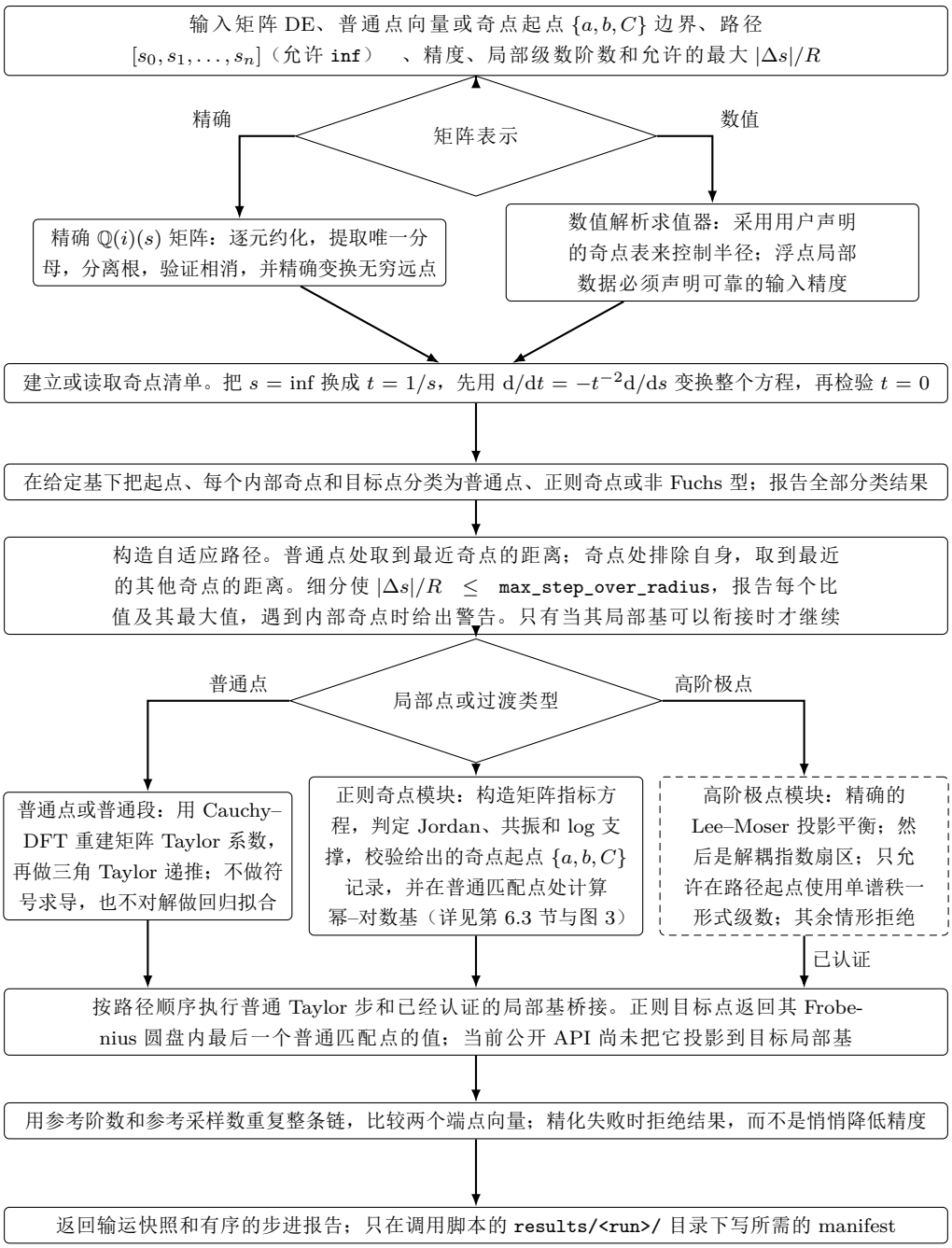


图 1: 基本的数值矩阵 DE 流程。输入格式和输出布局见第 6.6 节；精确/数值表示与普通点递推见第 6.1 节；奇点清单和无穷远点见第 6.1.2 节；路径构造和各 $|\Delta s|/R$ 报告见第 6.1.3 节；矩阵指标数据见第 6.2 节；对数节点见第 6.3 节；高阶极点模块及其拒绝边界见第 6.4 节；输运与精化见第 6.6.3 节。

3 核心功能二：半解析调节参数级数重建

第二个函数重建

$$F(\epsilon) = \sum_{n=n_{\min}}^{m} f_n \epsilon^n + O(\epsilon^{m+1}), \tag{2}$$

其中整数幂次是解析信息，每个 f_n 是任意精度的数值向量。m 和上游已做符号认证的 $n_{\min}$ 都必须给出；采样网格、工作精度和额外拟合的幂次在未指定时自动选取。每个调节参数采样都会调用图 1 的完整基本求解器；程序包里没有另一套独立的“拟合解”DE 引擎。

4 安装与依赖

运行时只要求 Python $\geq$ 3.10、`python-flint>=0.6` 和 `sympy>=1.12`。发布后从软件包索引安装：

```
python -m pip install FlintNDE
```

或者对下载的源码树安装一次：

```
python -m pip install path/to/FlintNDE
```

开发时的可编辑模式用同样的命令加 `-e`。此后调用脚本可以放在任意目录，只需 `import flintnde`；程序包源码不会复制到脚本旁边。调用方传入 `initialize_output_layout(__file__)` 时，所有生成文件都以该调用脚本为根，绝不写进已安装的程序包目录。导入程序包会把全局 Acb 上下文初始化为 200 位十进制工作精度加 32 个保护位；调用 `configure_working_precision()` 恢复该缺省值，显式给出的正整数则覆盖它。

每个 Python 进程中第一次 `import flintnde` 还会向 stderr 打印一条引用提示：“FlintNDE package paper, arXiv identifier pending.”。该提示不列出 dSIBP、MadStree、AMFlow 或任何其它文章。走文件协议的适配器可以在导入前设置 `FLINTNDE_SUPPRESS_CITATION_NOTICE=1`，并在响应元数据里保留自己的引用列表；这条提示绝不能混进机器可读的 stdout。

源码树还提供一个标准的 Wolfram Language 程序包。把含有 `FlintNDE.wl` 的目录加入 `$Path`，然后执行

```
Needs["FlintNDE`"];
system = FlintNDERationalSystem[{{1/(x-1)}}, x];
plan = FlintNDEPlanPath[system, 0, {1/2},
  "WorkingPrecisionDigits" -> 100];
result = FlintNDEExecutePath[system, {1}, plan,
  "WorkingPrecisionDigits" -> 100];
```

符号 `$FlintNDEVersion` 返回已加载程序包的版本串。规划阶段的缺省值是 `"SingularityMode" -> "Avoid"`；显式选择 `"SingularityJump"` 时程序会给出分支警告。这两个是唯一接受取值。消息缺省用英文，`MessageLanguage -> "CN"` 选择中文。

在显式的奇点跳越（singularity jump）模式下，精确的中间奇点用户点不是普通的输运节点。该目标点及其两侧落在同一收敛圆盘内的所有用户点，都由同一个局部幂–对数基负责；圆

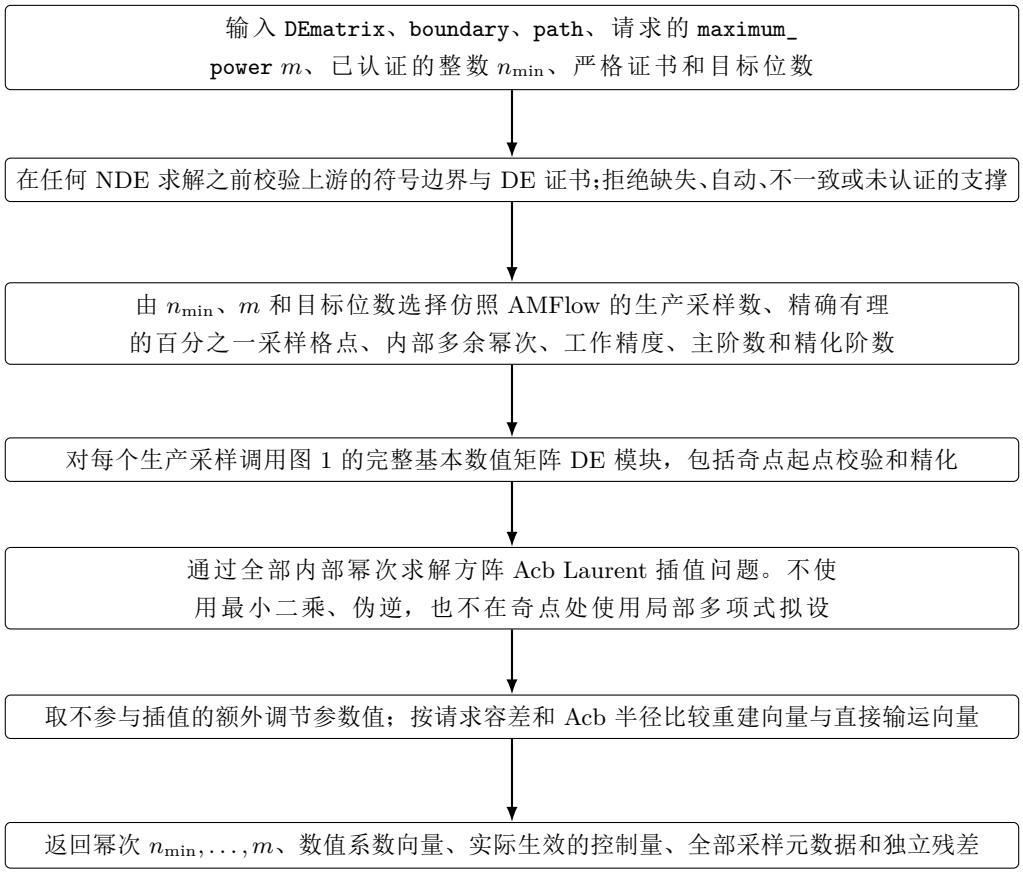


图 2: 半解析调节参数级数流程。基本 NDE 节点就是图 1，不再重复展开。AMFlow 误差估计、自动 ϵ 尺度、多余阶数、精度规则、支撑证书、方阵插值和验证详见第 6.5 节；完整可选参数表见第 6.6 节中的表 6。

盘半径取到最近其他奇点的距离。输运从离开该基之后的普通匹配点继续。若目标恰好是终结奇点，则使用 `plan_singular_target_match`：当前状态若在保守局部圆盘之外，规划器会沿入射方向插入一个隐藏的普通匹配点。报出的值来自低阶主链，高阶参考链只负责检查各分量的分类和有限值精度。在精确奇点处做不共线转向仍保持拒绝，因为这种输入没有确定延拓的分支。

这套目标点逻辑与坐标无关。候选点由完整的精确有理矩阵发现，目标点可以位于 $\mathbb{Q}(i)$ 中的任何位置；变量名和数值零都没有特殊地位。回归测试把目标移到 $2+\mathrm{i}$，同时在 $5+\mathrm{i}$ 保留另一个有限奇点，并从同一个二维系统中得到一个可去分量和一个真实极点分量。这可以防止通用入口依赖零点、单字母或特定拓扑的分类。

Wolfram 选项 `"WorkDirectory"` 指的是临时运行根目录本身。取 `Automatic` 时它在当前调用目录下建立一个 `results_temp/`，接口只在其下追加一个短的 `bridge/` 子目录，文件名含 16 位十六进制 token。它不会再追加第二个 `results_temp`，也不会建立冗长的任务专属目录。在 Windows 上，任何文件写入或 Python 启动之前都会检查全部完整的请求、输出和日志路径。路径超过 259 个字符时返回 `RuntimePathTooLong`，并给出实际路径和长度。请求写入失败、缺少 `python-flint`、桥接启动失败、零退出但缺少输出以及输出非法，分别报告为 `RuntimeInputWriteFailed`、`PythonFlintUnavailable`、`BridgeLaunchFailed`、`BridgeOutputMissing` 和 `BridgeOutputInvalid`。

固定调节参数的独立任务使用 `FlintNDEEvaluateEpBatch`，选项为 `ParallelTaskCount -> 12`。缺省上限是 12 个工作进程；实际生效数取该上限与任务数中的较小者，排队的任务会在有工作进程空出时自动开始。Python 侧的对应接口是 `run_ep_tasks(..., parallel_task_count=12)` 和 `reconstruct_series_solution` 的同名选项。这个外层进程数与 python-flint 的进程内 `ctx.threads` 设置互不相干。

当前版本目录还包含 `engine/` 下的独立 C++ 引擎，在 MSYS2 UCRT64 环境中针对 FLINT 3.x 开发库用 `make` 构建；不分发预编译的二进制文件。可执行程序的调用方式为

```
flintnde_engine INPUT.json OUTPUT.json [--workers N]
                [--flint-threads N] [--residues auto|sparse|dense]
```

它的接口约定和 Wolfram 直接入口见第 6.6.6 节。

完整的输入语法和全部可选参数见第 6.6 节。

5 例子

5.1 带认证的 Laurent 重建

可执行脚本 `examples/ep_series_reconstruction.py` 使用零矩阵 DE 和解析已知的边界

$$Y(0,\epsilon)=\epsilon^{-1}+2+3\epsilon+4\epsilon^{2}+5\epsilon^{3}+6\epsilon^{4}. \tag{3}$$

因此上游例子证书声明 $n_{\min}=-1$。FlintNDE 在任何求解之前校验该证书，但不从采样值反推它。用户只请求有限部分（$m=0$）；第一次方阵拟合额外包含两个幂次。由于独立点容差被刻意取严，第一轮失败后会追加两个生产采样点，同时复用此前所有生产和验证数值。最终接受的结果恢复了极点系数 1 和有限部分 2。该例子使用 `parallel_task_count=12`；当调节参数任务数不足时，进程池自动减小实际并发数，多余的任务进入队列。

5.2 de Sitter 后端用法

MadStree 在上游层提供物理的 de Sitter 例子：它导出符号边界和 dlog DE，认证调节参数支撑，然后调用本程序包。FlintNDE 不在一份“只有后端”的例子里重复 dS 的拓扑、主积分顺序或归一化。

5.3 Schwarzschild 双端径向系统

保留下来的跨领域例子从奇宇称 Regge–Wheeler 函数 u_o^- 出发，单位取 $2M=1$：

$$\frac{\mathrm{d}^2u_o^-}{\mathrm{d}x^2}+\frac{1}{x(x-1)}\frac{\mathrm{d}u_o^-}{\mathrm{d}x}+\left[\frac{\omega^2x^2}{(x-1)^2}-\frac{\ell(\ell+1)x-3}{x^2(x-1)}\right]u_o^-=0. \tag{4}$$

这里不必再为偶宇称函数 u_o^+ 单独指定方程：在固定 (ℓ,ω) 下，它通过标准的 Chandrasekhar–Darboux 变换 [20] 由 u_o^- 及其一阶导数给出。前因子

$$u_o^-=x^{\alpha_0}(x-1)^{\alpha_1 i\omega}u,\qquad(\alpha_0,\alpha_1)=(3,-1), \tag{5}$$

消去二阶有限极点。记 $U=(u,u')^T$，用

$$S_\infty=\begin{pmatrix}1 & 1\\ i\omega & -i\omega\end{pmatrix},\qquad U=S_\infty a, \tag{6}$$

把常数部分对角化，就得到显式整理好的一阶系统

$$\frac{\mathrm{d}a}{\mathrm{d}x}=\left[\Lambda_\infty+\frac{R_0}{x}+\frac{R_H}{x-1}\right]a,\qquad\Lambda_\infty=\begin{pmatrix}i\omega & 0\\ 0 & -i\omega\end{pmatrix}, \tag{7}$$

其中，记 $\lambda=(\ell-1)(\ell+2)/2$、$d=(\lambda-2)/\omega$，

$$R_0=\begin{pmatrix}-5+id & id\\ 5-id & -id\end{pmatrix}, \tag{8}$$

$$R_H=\begin{pmatrix}2+2i\omega-id & 3-id\\ -2-2i\omega+id & -3+id\end{pmatrix}. \tag{9}$$

于是这个例子不需要未经约化的伴随矩阵写法。

前导级数系数为零的物理含义，要在除掉已知端点因子之后再读。视界处 $r_*\sim\log(x-1)$，入射平面波 $e^{-i\omega r_*}$ 已被式 (5) 吸收，剩下的 Frobenius 级数指标前导幂为零。无穷远处，在提出 $e^{\pm i\omega r_*}$ 与 $z=1/x$ 中相应的代数幂之后，出射或入射的解析余项同样从零次幂开始。这些零根标记的是渐近平面波的正则余项，而不是物理上的常数波。

对表中给出的准正规模频率，脚本分别在 $x=1$ 和 `inf` 各起算一次，把选定分支输运到共同匹配点，再把每个向量在对面端点的基下分解 [21, 22]。“禁止分量与允许分量”之比为

频率	视界 $\to$ 无穷远	无穷远 $\to$ 视界
表中准正规模频率	2.4549×10^{-15}	3.7235×10^{-15}
相对偏移 10^{-3}	3.9002×10^{-3}	5.9175×10^{-3}

无穷远先映射到 $z=1/x=0$。其 A_{-2} 本征值简单且互异，因此式 (36) 可以构造出射与入射分支；最近根间隙规则取 $q=3$，给出第一个普通点 $x\simeq 74.20$。在请求的 38 阶下，出射/入射的五阶块比值约为 2.9034×10^{-3} 和 3.1998×10^{-3}；在可用范围内，两个分支都还没到最小项。它们的局部向量与独立实现的标量二阶递推分别相符到 3.3×10^{-36} 和 9.4×10^{-36}；另在 $x=28$ 处比较，仍优于 3×10^{-18}。起点更远会放大刚性指数分支之间的 Cauchy–DFT 混叠，因此该例子取采样半径分数 0.40；此时主链与参考链的最大输运差为 3.75×10^{-22}。于是表格第一行在所展示的精化尺度上与零一致，而非零的偏移行则是反事实的边界失配。这是形式/Gevrey 的无穷远展开，不是收敛的 Frobenius 圆盘。该例子只检验径向连接问题，不引入任何积分族假定。

同一脚本的第三段只用视界起点，并按同样的 10^{-10} 相对精度给出该模式的完整径向剖面。采样点在乌龟坐标 $r_*=x+\ln(x-1)$ 上于 $[-6,160]$ 内均匀分布（若用均匀 x，振荡会在视界附近被无限压缩），再用 Newton 迭代反解出 $x>1$，另外加入精确的 $x=4$：共 721 个点。一次规划就能覆盖它们，步长分数 0.06 时共 271 个节点；585 个点是共享所在段 Taylor 系数的内部采样点，最大的一桶装 32 个点。两条链的端点差为 1.5×10^{-17}，而 $x=4$ 处的值与独立规划的视界链相符到 5.2×10^{-32}。收紧步长分数是必要的：剖面的增长大致像 $\mathrm{e}^{\mathrm{Re}(w)r_*}$，在 $r_*=160$ 处达到 $|u_o^-|\simeq 2.8\times10^{12}$，因此沿链注入的截断误差会被剩余增长放大；若采用缺省规划上限 1/4，在 38/55 阶下剖面末端的双链比较会直接失败。曲线采样写入 `transport/qnm_2x2_curve_samples.json`。

6 技术细节

6.1 矩阵微分方程与普通点

采用列向量约定

$$\frac{\mathrm{d}}{\mathrm{d}z}Y(z)=A(z)Y(z),\qquad A(z)\in\mathbb{C}^{N\times N}. \tag{10}$$

在普通点 c 处令 $s=z-c$，写

$$A(c+s)=\sum_{m\ge0}A_m(c)s^m,\qquad Y(c+s)=\sum_{n\ge0}Y_n(c)s^n. \tag{11}$$

代入式 (10) 得到三角递推

$$Y_{n+1}(c)=\frac{1}{n+1}\sum_{m=0}^{n}A_m(c)Y_{n-m}(c). \tag{12}$$

递推可以推进一个边界向量，也可以推进完整的基矩阵。只需要一个物理边界条件时，前者代价更低。

6.1.1 一般解析矩阵系数

缺省后端并不假定 $A(z)$ 只有单极点。若 A 在圆 $|s|=r$ 内及边界上解析，Cauchy 公式给出

$$A_m(c)=\frac{1}{2\pi r^m}\int_0^{2\pi}A(c+re^{\mathrm{i}\theta})e^{-\mathrm{i}m\theta}\,\mathrm{d}\theta. \tag{13}$$

取 K 个等距采样点后，它就变成离散 Fourier 投影

$$A_m^{(K)}(c) = \frac{1}{Kr^m} \sum_{j=0}^{K-1} A(c + r\xi_K^j)\xi_K^{-jm}, \qquad \xi_K = e^{2\pi \mathrm{i}/K}. \tag{14}$$

这是从数值矩阵取值重建系数，既不是符号求导，也不是对解做回归拟合。只要采样圆落在它们的公共解析域内，它对有理矩阵、代数矩阵和用户提供的解析矩阵求值器同样适用。

对有理矩阵，以后也可以把直接级数除法作为可选的系数生成器。式 (14) 仍是一般的缺省做法，也是常用的校验路线。截断与 Fourier 混叠误差由第二条输运监控：后者使用更高的 Taylor 阶数和更多的 Cauchy 采样点。

6.1.2 精确奇点清单与无穷远点

输入为精确 Gaussian 有理矩阵时，先把每个矩阵元在 $\mathbb{Q}(i)[s]$ 中约化：

$$A_{ij}(s) = \frac{N_{ij}(s)}{D_{ij}(s)}, \qquad \gcd(N_{ij}, D_{ij}) = 1. \tag{15}$$

有限候选点集合取所有约化分母的根之并。标量 $a + ib$ 存成两个 FLINT `fmpq` 值，N 维复矩阵存成两个 N 维 `fmpq_mat` 对象；不使用 $2N$ 维实分块嵌入。精确的 $\mathbb{Q}(i)[s]$ 算术负责多项式 gcd 与无平方分解。若无平方因子同时包含精确根零和 $\mathbb{Q}(i)$ 之外的代数根，则先除出因子 s。这样边界点 $s = 0$ 保留其精确 Gaussian 有理身份，仍可进入精确 Frobenius 调度；其余根则作为 Acb 球分离出来。某个根处记录的极点阶数，取完整矩阵所有元素中的最高阶。完整矩阵的一阶极点属于 Fuchs 型；阶数大于一只能证明 `non_fuchsian_input_basis`，它本身并不是奇点不规则性的基无关证明。特别地，只看对角元是错的；但高阶的非对角元仍可能通过亚纯规范变换或剪切变换降阶。先合并、约化每个完整矩阵元，再收集分母，这一步是必需的：分母相同的项可能相消，被消掉的候选点不能进入清单。

无穷远点用它自己的局部变量检验。取

$$t = \frac{1}{s}, \qquad \frac{\mathrm{d}s}{\mathrm{d}t} = -\frac{1}{t^2}, \tag{16}$$

式 (10) 变成

$$\frac{\mathrm{d}Y}{\mathrm{d}t} = B(t)Y, \qquad B(t) = -\frac{1}{t^2} A\left(\frac{1}{t}\right). \tag{17}$$

因此，完整 B 在 $t = 0$ 处的极点阶数（已含求导 Jacobian）判定无穷远点在变换后的输入基中属于普通点、Fuchs 型还是非 Fuchs 型。对分子分母次数分别为 n 和 d 的已约化标量矩阵元，其贡献的极点阶数为 $\max(0, n - d + 2)$。精确整除比用小数值 δ 试探 $A(p + \delta)$ 既快又强；位移求值是有用的诊断，但既不能证明相消，也不能把很小的留数与零区分开。

建好这份清单后，程序包尝试精确的结构分解

$$A(s) = P(s) + \sum_j \frac{R_j}{s - p_j}, \tag{18}$$

其中矩阵多项式 P 的次数可以任意有限。极点位置和留数在内部导出，调用方不必提供 dlog 字母表。只有当完整矩阵能被精确重建、且每个有限极点都是单极点时，才选用这个部分分式递推；出现高阶极点或任何恒等式校验失败，就留在一般的有理矩阵路线上。纯多项式系统是零有限极点的合法子情形。因此式 (18) 是对通用接口的认证优化，而不是把适用范围限制在 dlog 微分方程。

6.1.3 路径细分与精度

在普通点 c 处，记 $R(c)$ 为到最近有限奇点的距离。在奇点 p 处定义

$$R_{\mathrm{sing}}(p) = \min_{q \neq p} |p - q|, \tag{19}$$

即排除该点自身。程序包对每一段都使用

$$|\Delta z| < r < R(c). \tag{20}$$

长路径按 $R(c)$ 的固定比例细分。主链和参考链独立求值。用户可见的端点值、稠密采样、保存点和调节参数拟合采样都取自阶数较低的主链；高阶参考链只用于最后的相对差诊断，而这个诊断与单个 FLINT 球自带的半径不是一回事。多段计算从高精度中点重新开始，以限制区间包裹，因此被报告为任意精度点输运，而不是对整条路径的严格包围。

收敛半径和步长是两个不同的量。奇点集合决定 $R(c)$ 或 $R_{\mathrm{sing}}(p)$；用户选项 `max_step_over_radius` 只施加约束 $|\Delta z|/R \le f_{\mathrm{step}}$。规划与执行是分开的操作：规划器读取用户给出的原始点，序列化所有普通节点、采样分配、奇点分类和桥接几何；执行器恢复该规划，不得再次调用规划器。

缺省模式是 `singularity_mode="avoid"`。直线段内若含内部奇点，规划会停止，并给出结构化报告，指出该奇点和受影响的用户点对。调用方也可以显式给出绕行点。凡穿过中间节点的多点输运，本手册统称跳越（jump）；只有显式的 `"singularity_jump"` 模式才会跨越奇点构造局部基桥接，这种操作称为奇点跳越。不接受布尔开关，也不接受其它拼写。奇点跳越相当于在多个绕行同伦类中选定一个分支，程序包会把这一责任报告出来，由调用方确认分支。桥接可以使用原始 Frobenius 系统、Moser 约化后的 Fuchs 型系统，或经过认证的指数乘幂乘对数扇区。若这些精确门禁都不能给出局部基，高阶极点检查点仍保持拒绝。

启用奇点跳越后，进入某个极点的步长范围时会生成入射和出射两个匹配节点。其后的用户点按顺序检查；仍落在一个可允许的、以该极点为中心的步长内的最后一个点，成为下一个节点。若第一个点已在该范围之外，就先朝它走一个可允许的步长，再恢复普通规划。入射步长由目标奇点的 $R_{\mathrm{sing}}(p)$ 控制，而不是由前一个普通点的半径控制。

全局 FLINT 精度为

$$p_{\mathrm{bit}} = \lceil p_{\mathrm{dec}} \log_2 10 \rceil + 32. \tag{21}$$

例如 70 位和 100 位十进制精度分别要求 265 和 365 位，更高的十进制精度继续按同一公式增加位数。段投影、匹配比、旋转因子以及绕数/单值几何都在当前 Arb 精度下构造，而不是在 binary64 下构造。序列化的规划记录 `planning_precision_digits`；Acb 坐标存成 Arb 的中点–半径–指数记录。经 Wolfram 桥接执行时，若工作精度高于规划精度则被拒绝并要求重新规划，因为序列化无法恢复已丢弃的信息；Python 的直接“恢复并执行”入口按当前全局精度运行。只增加输出位数不改变工作精度。Python 导入和全部四个 Wolfram 入口都缺省使用 200 位十进制精度，含保护位共 697 位。

表 1: 普通路径点与奇点路径点的执行约定。

路径角色	实际执行的动作、半径规则与返回对象
普通起点	输入是有限 Acb 列。Taylor 输运从给定点开始，其收敛半径为到最近奇点的距离。
奇点起点	正则奇点接受精确的 $\{a, b, C\}$ 幂–对数记录；指数型奇点只接受显式的 $\{\phi, a, b, C\}$ 记录。两者都不是奇点处的有限值。程序包校验指标、指数和对数分支，在第一个普通匹配点求值，然后开始 Taylor 输运。匹配距离受 R_{sing} 限制，该半径排除奇点自身。对指数解，FlintNDE 从 DE 导出精确扇区并验证 ϕ，绝不从 C 反推缺失的指数特征。
普通目标点	最后一个快照就是给定目标处的 Acb 列。
正则奇点目标	可执行路径终止于目标 Frobenius 圆盘内的一个普通匹配点，绝不在奇点处臆造有限值。报告目标分类、奇点、匹配点和匹配距离比。若目标坐标带有 save 标记且可直接构造正则奇点幂–对数基，FlintNDE 求出并写出可复用的 $\{a, b, C\}$ 系数；它们是局部边界数据，不是奇点处的有限值。
内部正则奇点	路径在两侧各存一个普通匹配点，二者之间是 Frobenius 桥接。两个“匹配点到奇点”的距离都除以奇点半径 R_{sing}，而不除以任一相邻普通点的半径。报告主 Acb log/幂分支以及两个比值。
高阶极点检查点	清单先记录 `non_fuchsian_input_basis`。局部调度器先试精确 Lee–Moser 投影平衡，再试经过认证的解耦指数扇区门禁。成功时使用同一个两侧局部桥接，并返回原始积分基。否则延拓被拒绝；内部点可以改用显式的普通绕行点绕开。标有 `(coordinate,"save")` 且可衔接的指数检查点，会在桥接处写出可复用的 $\{\phi, a, b, C\}$ 数据。
无穷远点	完整方程先变换到 $t = 1/s$，含 $-t^{-2}$ Jacobian。得到的 $t = 0$ 随后遵循同样的普通点、正则奇点或非 Fuchs 输入基规则；其局部半径在变换后的变量中量到最近的其他奇点。

6.1.4 能力预检与拒绝边界

输运之前，`build_adaptive_path_plan` 对两个端点和每个内部奇点检查点施加同一个局部调度器。它的 `continuation_ready` 标志是能力门禁，不是精度估计。若该标志为假，规划会记录奇点分类、尝试过的认证路线以及缺失的结构（例如分支或亏损的形式块、代数扩张、Stokes 连接）。此时调用方必须选择普通绕行点，或提供外部的局部连接数据；不得在该规划上启动数值输运。

这个门禁作用于每个非普通检查点，而不只是高阶极点。中心不在 $\mathbb{Q}(i)$ 中的正则奇点，或者其精确指标、Jordan、共振结构不被当前幂–对数构造器支持的正则奇点，同样会置 `continuation_ready=False`。直接构造路径时，程序会报告点的类型和局部基失败的原因，而不是把失败推迟到输运阶段。

直接调用路径构造器也遵守同一规则：它在输运前抛出 `LocalReductionError`。具体而言，精确门禁、Lee–Moser 门禁、指数门禁或秩一形式门禁失败时，绝不回退到普通 Taylor/Cauchy 展开。唯一的“只在起点”形式路线，是在单个 Stokes 扇区内从奇点边界初始化到第一个普通匹配点的解；若同一点出现在内部或作为目标，它就不是可衔接的。

6.1.5 只按坐标的 save 标记

被保存的路径点没有用户给的名字，其输入严格是 `(coordinate, "save")`，例如

```
path = [(z0, "save"), z1, (z2, "save")]
transport_path_refined(system, boundary, path)
```

同一对写法也可用于自适应路径的起点、目标或绕行坐标。奇点作中间坐标时只接受这种标记，它表示保存一个奇点检查点，而不是普通绕行。内部的 `NamedPoint` 标签只用于诊断路由，不属于保存约定。

到达带标记的坐标时，FlintNDE 立即按路径顺序写出 `flintnde_save_001.json`、`flintnde_save_002.json` 等等。缺省目录是调用方的 `Path.cwd()`；`save_output_directory` 可以另选一个归调用方所有的目录。只有全部输运步骤都成功后，程序包才写 `flintnde_save_points.json`。这样后续失败时，已完成的点文件仍然保留，但不会声称汇总完整。精化输运只写主链；参考链始终只是精度诊断，绝不替换用户结果。

在普通点，记录含坐标和输运后的 Acb 列。在直接的正则奇点起点或目标，记录含可复用的 $\{a, b, C\}$ 项和 `resultType=frobenius_boundary`。经过认证的解耦指数起点、目标或中间桥接，则记录 $\{\phi, a, b, C\}$ 和 `resultType=exponential_boundary`。这些都是边界数据，不是 $Y(z_*)$。只在起点的形式指数分支可以在起点保存，并记录 `continuationReady=false`；同一方法用在内部或目标处仍然禁止。保存经过 Lee–Moser 变换的原始基奇点，仍要求有认证的局部逆射流；不支持的 Stokes 连接保持拒绝。

6.2 正则奇点与矩阵指标方程

在 $z = 0$ 附近考虑 Fuchs 型系统

$$\frac{\mathrm{d}Y}{\mathrm{d}z} = \left(\frac{R}{z} + \sum_{m \geq 0} A_m z^m \right) Y. \tag{22}$$

代入 $Y = z^\rho \sum_{n \geq 0} v_n z^n$，前导方程为

$$(\rho \mathbf{1} - R) v_0 = 0, \qquad \det(\rho \mathbf{1} - R) = 0. \tag{23}$$

式 (23) 就是矩阵指标方程。确定局部指数时，不需要先把系统消元成标量高阶方程。对半单且无共振的分支，高阶系数满足

$$\big((\rho + n)\mathbf{1} - R\big) v_n = \sum_{m=0}^{n-1} A_m v_{n-1-m}. \tag{24}$$

广义级数及其延拓在费曼积分中的实现可参考文献 [23]。

6.3 何时需要对数

指标根重复本身不是出现对数的判据。真正有两种不同机制。

6.3.1 非半单留数

设某个单根块为 $R=\rho\mathbf{1}+N$，其中 N 幂零。仅在前导系统中就有

$$z^{R}=z^{\rho}e^{N\log z}=z^{\rho}\sum_{q=0}^{\nu-1}\frac{N^{q}}{q!}(\log z)^{q}, \tag{25}$$

其中 $N^{\nu}=0$。因此长度为 ν 的 Jordan 块会产生到 $\nu-1$ 次的对数。相反，重复但半单的本征值有 $N=0$，这条机制不产生对数。

6.3.2 整数差的共振

即使留数可对角化，只要 $\rho+n$ 是另一个指标根，式 (24) 就会奇异。记 $P_{\rho+n}$ 为到该本征空间的精确谱投影，b_n 为右端项。纯幂递推存在，当且仅当

$$P_{\rho+n}b_n=0. \tag{26}$$

若该相容性条件不成立，缺陷会转交给对数系数。程序包用精确有理矩阵计算式 (26)。缺省路线绝不用浮点阈值悄悄替代这个精确判定；下面描述的显式数值路线会完整记录其阈值来源。

6.3.3 统一的幂–对数递推

记

$$Y=z^{\rho}\sum_{n\ge 0}\sum_{q=0}^{q_{\max}}v_{n,q}z^{n}(\log z)^{q}. \tag{27}$$

固定 (n,q) 时，递推形如

$$\big((\rho+n)\mathbf{1}-R\big)v_{n,q}+(q+1)v_{n,q+1}=\sum_{m=0}^{n-1}A_{m}v_{n-1-m,q}. \tag{28}$$

缺省实现用 FLINT 有理矩阵生成一份精确清单，包含根、Jordan 幂零性、谱投影、初始本征向量以及每一个生效的共振门禁；随后 FLINT 在请求精度下计算式 (28)。精确分类是缺省做法，精确判定失败后绝不回退到阈值判定。只有浮点输入才走数值门禁，它使用

$$\tau_{\mathrm{rel}}=N\,10^{-p},\qquad \tau_{\mathrm{abs}}=\tau_{\mathrm{rel}}\max\left(\max_{i,j}|R_{ij}|,\max_{m,i,j}|(A_m)_{ij}|\right), \tag{29}$$

其中 N 是系统维数（保守的秩上界），p 是可靠的十进制输入精度，而不只是算术工作精度。Python 的 binary64 输入上限取 15 位十进制。更高精度的十进制或 Acb 输入必须自带或声明其可靠精度，否则分类拒绝。用户可以替换 τ_{rel}。清单和强制警告会报告矩阵尺度、两个阈值，以及被置零的最大量和被保留的最小量（既给绝对单位，也给相对于矩阵尺度的值）。矩阵尺度为零时会明确报告，并给出未定义的相对比值，而不是编造数字。这条阈值路线明确是数值的，不是精确证书。含混合根的亏损系统若有歧义，一律拒绝。

图 3 汇总了实际实现的判定路线。非半单机制与共振机制是叠加的：Jordan 对数不会跳过其后的共振检查。

只要任一机制启用了对数扇区，局部解就一律由式 (28) 生成。程序包没有那种“对纯多项式解做回归、从而丢掉对数分支”的奇点路径。第 6.5 节的 Laurent 拟合只涉及调节参数依赖，不是局部解拟设。

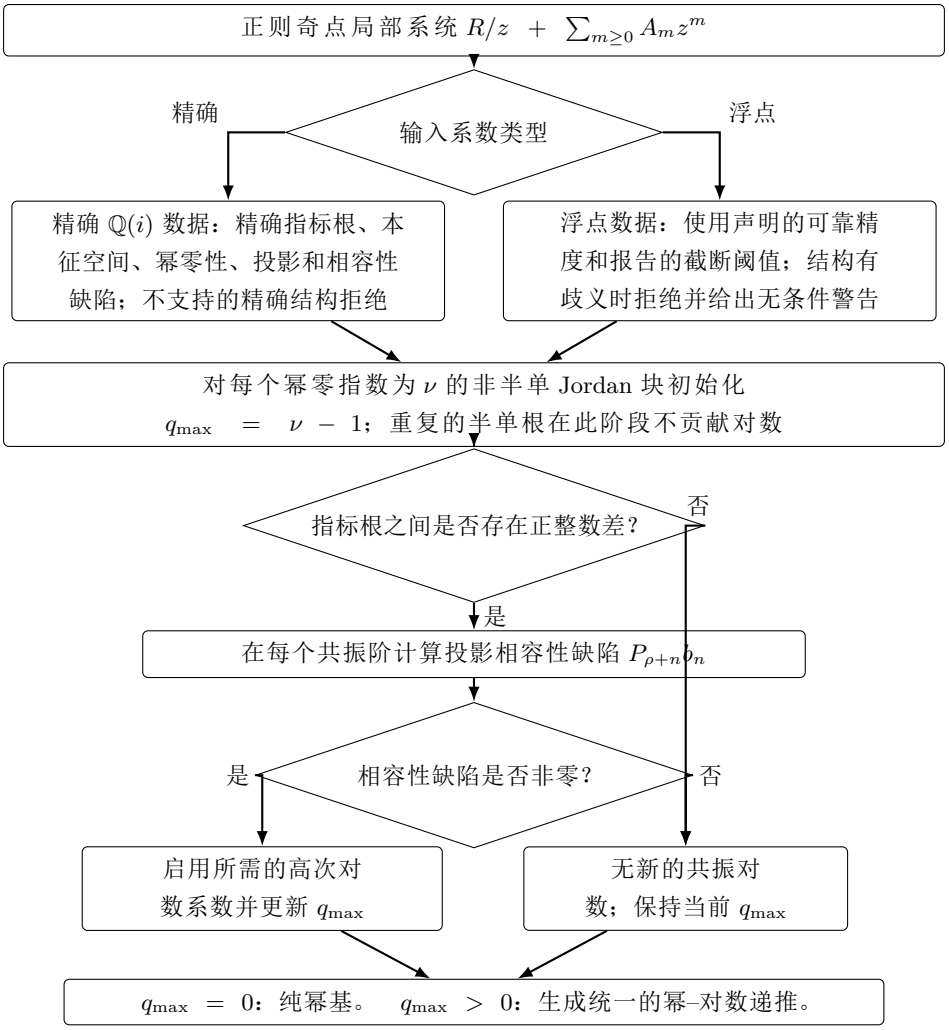


图 3: 判定对数支撑的精确优先路线与考虑精度的数值路线。

6.4 非 Fuchs 型输入基与真正的不规则性

检验必须用完整矩阵，不能只看对角元。但即使如此，在任何元素中发现二阶或更高阶极点，也只能证明给定基是非 Fuchs 型。亚纯规范变换（包括剪切变换）可以降低极点阶数，也可以消掉高阶的非对角项。因此 FlintNDE 采用下面的三阶段局部调度器；清单上的标签本身仍按保守方式记录。

先做精确的 Lee–Moser 约化。在阶数为 $p+1>1$ 的极点处写

$$A(z) = z^{-p-1}\big(A_0 + zA_1 + O(z^2)\big). \tag{30}$$

程序包构造 A_0 的精确幂零 Jordan 链，用 A_1 定出链投影，并构造

$$T_Q(z) = \mathbf{1} - Q + \frac{Q}{z}, \qquad T_Q^{-1}(z) = \mathbf{1} - Q + zQ, \qquad Q^2 = Q. \tag{31}$$

对 $I = T_Q J$，完整的变换后矩阵为

$$A_{\text{new}} = T_Q^{-1} A T_Q - T_Q^{-1} T_Q'. \tag{32}$$

所有运算都在 $\mathbb{Q}(i)$ 上完成。只有当幂等性精确成立、且完整变换后矩阵的极点阶数严格下降时，才保留这次平衡。随后重新计算前两个 Laurent 矩阵，重复该过程，直到系统成为 Fuchs 型。清单保存有序的投影和完整的极点阶数历史；对最终系统施加全部逆平衡，必须逐元精确复现原来的有理矩阵。最高的 Laurent 矩阵若非幂零，就是这个无分支亚纯类中通往 Fuchs 规范的精确障碍。内部构造假设若不成立，则报告为“不确定”，而不是报告为真正的不规则性。

局部幂–对数解在约化后的基中求值，返回前再乘上完整的乘积 $T_{Q_1}\cdots T_{Q_m}$。奇点起点 $\{a, b, C\}$ 记录直接在原始积分基中校验：FlintNDE 生成约化 Frobenius 基的精确有限射流，把整个射流乘上 Laurent 规范，将更早的幂次和更高次对数置零，再与请求的系数 C 匹配。这一步是必要的，因为非对角平衡之下，相邻的级数系数也会贡献到原始基的前导向量。

若仍有高阶极点，FlintNDE 尝试一条受限的指数路线。假定存在某个精确的 $\mathbb{Q}(i)$ 常数基，使所有更高的 Laurent 矩阵都成为对角矩阵，并把系统分成高阶对角特征相同的若干扇区。对其中一个扇区写

$$A_{\text{high}}(z) = \sum_{m=2}^{p} \frac{c_m}{z^m}\,\mathbf{1}. \tag{33}$$

取 $Y = e^{\Phi(z)}U$，直接代入得到

$$\Phi'(z) = \sum_{m=2}^{p} c_m z^{-m}, \qquad \Phi(z) = -\sum_{m=2}^{p} \frac{c_m}{m-1} z^{-(m-1)}. \tag{34}$$

剩余的 U 方程极点阶数至多为一，此后原封不动地使用普通的幂–对数机制。特别地，$A_{-2} = k$ 给出 $e^{-k/z}$。算出的 $k = 0$ 只说明这个标量指数不存在，并不能解除幂零、亏损或非对角的高阶极点约束；这种块仍须通过 Moser 约化或更一般的形式约化。

精确解耦门禁仍刻意保持严格：它拒绝亏损的前导矩阵、$\mathbb{Q}(i)$ 之外的本征值、所选基中的非对角高次 Laurent 矩阵、不同指数特征之间的耦合，以及相减后仍然残留的高阶极点。边界仍用 $\{a, b, C\}$ 表示幂–对数因子。变换后的精确 C 自动选定指数扇区；若一个项跨越多个扇区，用户必须把它拆成多项。

还有一条显式的形式路线。极点阶数恰好为二且 A_{-2} 在 $\mathbb{Q}(i)$ 中有 N 个简单互异本征值 k_j 时，对每一对右/左本征向量 (v_0, ℓ)，FlintNDE 构造

$$Y = e^{-k/z} z^{\rho} \sum_{n=0}^{N_{\mathrm{form}}} v_n z^n, \qquad \rho = \frac{\ell^T A_{-1} v_0}{\ell^T v_0}. \tag{35}$$

令 $z^{\rho+q-2}$ 的系数相等，得到

$$(k\mathbf{1} - A_{-2}) v_q = [A_{-1} - (\rho + q - 1)\mathbf{1}]\, v_{q-1} + \sum_{m=0}^{q-2} A_m v_{q-2-m}. \tag{36}$$

在 $q+1$ 阶上，左本征向量的相容性条件定出 v_q 中本来自由的 v_0 分量。所有本征向量、ρ、相容性检验和递推求解都在 $\mathbb{Q}(i)$ 中精确进行；不使用数值秩截断、最小二乘或多项式回归。公开的边界仍是 $\{a, b, C\}$：目前 $b = 0$、$a = \rho$，精确向量 C 必须选定一个前导本征方向，用户不需要另外输入 k。

出现 $e^{-k/z}$ 本身并不意味着发散。在解耦路线中，扣掉完整的标量高阶极点部分之后，残差系统的极点至多是一阶，给出收敛的 Frobenius 幂–对数级数，其圆盘由最近的其他奇点界定。相反，式 (36) 的递推一般只是扇区性的 Gevrey/渐近级数。FlintNDE 总是算到请求的阶数 N，并额外生成五个系数向量用于诊断；它不会悄悄用观测到的最小项阶数替换 N。记 $T_n = v_n z^n$、$S_N = \sum_{n=0}^{N} T_n$，报告的五阶块比和相对精化为

$$r_5 = \frac{\left\| \sum_{j=1}^{5} T_{N+j} \right\|_{\infty}}{\left\| \sum_{j=0}^{4} T_{N-j} \right\|_{\infty}}, \qquad \delta_5 = \frac{\|S_{N+5} - S_N\|_{\infty}}{\|S_{N+5}\|_{\infty}}. \tag{37}$$

五个向量项先相加，再取无穷范数。期望的“块递减”条件是 $r_5 < 1$。条件不满足并不会压掉结果：FlintNDE 给出醒目的警告并记录该问题。实际值总会被报告，即使它小于一，用户也能判断递减是否足够。报告还保留最小项的阶数和量级作为辅助信息，并说明是否达到了那个阶数。若调用方手上已有标量项，公开函数 `five_term_tail_diagnostic` 改为计算

$$r_{5,\mathrm{scalar}} = \frac{\left| \sum_{j=1}^{5} T_{N+j} \right|}{\sum_{j=0}^{4} |T_{N-j}|}. \tag{38}$$

返回的 `FiveTermTailDiagnostic` 给出分子、分母、比值、调用方给的阈值和严格通过标志。这个标量诊断不替代式 (37)；分母为零，或区间比较无法证明严格不等式，都拒绝。费曼积分微分方程中的广义局部展开可参考文献 [23]。

于是“在不规则点附近能算多远”有两个不同答案。残差为 Fuchs 型的情形用 $|z| < R_{\mathrm{sing}}$ 和通常可配置的步长分数。形式渐近分支没有收敛半径；自动生成路径时，FlintNDE 用指数根之间的间隔。对扇区 i 定义

$$\Delta_i = \min_{j \neq i} |k_i - k_j|, \qquad \widehat{n}_{\min,i}(z) \simeq \frac{\Delta_i}{|z|}. \tag{39}$$

这是与最近竞争指数作用相对应的标准晚期项估计。若 N 是请求的形式级数阶数，$q > 1$ 是安全因子，建议距离为

$$|z|_{\mathrm{root}} = \min_i \frac{\Delta_i}{qN}. \tag{40}$$

缺省 $q=3$，也就是 N 约为估计最小项阶数的三分之一。根多于两个时，每个分支取它最近的另一个根，再取最小建议距离。单根标量高阶极点扇区由精确解耦的指数幂–对数路线负责，不用这条形式根间隙规则。路径区间和 `max_step_over_radius` R 只能进一步缩短距离，后者只在给出更小步长时生效。最终是否接受仍取决于式 (37) 的实际检验；根间隙估计不是收敛性或精度的证明。

因此到最近其他奇点的距离只是参考尺度，不是收敛半径。用户必须停留在一个 Stokes 扇区内，并检验阶数和匹配点的稳定性。用户给出的首批匹配点同样得到五阶报告和警告。故形式路线只允许作为路径起点；用在内部或目标处一律拒绝，因为 FlintNDE 没有可用于连接两侧的 Stokes 矩阵。

AMFlow 为这个问题中可约化的部分提供了具体的工程参考 [4, 5]。它的 DE 求解器先把矩阵分块；对每个对角块，用前两个局部系数迭代构造投影并施加平衡变换，直到 Poincaré 秩为零，然后用剪切变换把留数本征值规范化。剩下的非对角高阶块通过求解线性矩阵方程约化，之后把变换后的边界交给奇点 Frobenius 求解器。若前导矩阵有非零的 Jordan 本征值，投影路线就判定正 Poincaré 秩不可约并中止。FlintNDE 现在在完整矩阵上实现了精确的局部投影平衡迭代，包括有序投影和完整的有理系统往返检验。它不照搬 AMFlow 的全局分块、留数规范化或单独的非对角块求解器；那些是归一化和全局组织层面的工作，超出了“给定的极点能否化为 Fuchs 型”这个局部问题。

更一般的不规则系统需要形如

$$Y(z)=e^{\Phi(z)}z^{\rho}\sum_{n,q}v_{n,q}z^{n}(\log z)^{q},\qquad \Phi(z)=\sum_{j\geq 1}\phi_{j}z^{-j}. \tag{41}$$

的解。已实现的收敛标量扇区情形是式 (34)；只在起点的简单谱情形是式 (36)。重复/亏损的前导根、更高的 Poincaré 秩、分支、代数扩张、一般的形式块规范以及 Stokes 矩阵都未实现。凡需要这些数据，调度器一律拒绝；它绝不接受用“令 $k=0$”来冒充缺失的约化。

6.5 调节参数中的数值重建

许多维数正规化计算需要

$$F(\epsilon)=\sum_{n=n_{\min}}^{n_{\max}}f_{n}\epsilon^{n}+O(\epsilon^{n_{\max}+1}) \tag{42}$$

的系数。与其把每个中间表达式都按 ϵ 截断，不如在若干小的精确有理值 ϵ_j 处求解完整的数值问题，再重建最终的 Laurent 系数。AMFlow 就采用这一策略 [4]。

若目标系数阶数为 m，$N+1$ 个采样点位于半径 r 处，最近未被计入的奇点在尺度 R 处，AMFlow 分析中引用的估计是

$$E_{n}\sim\left(\frac{r}{R}\right)^{N+1-n},\qquad N\sim m-1+\frac{\log E}{\log(r/R)}. \tag{43}$$

若单次求解到误差 p 的代价为 $t(p)\sim(-\log p)^{\alpha}$，参考的优化选择是

$$r\sim RE^{1/(\alpha m)},\qquad N\sim(\alpha+1)m-1,\qquad p\lesssim E^{(\alpha+1)/\alpha}. \tag{44}$$

这些是指导性公式，不是事后证明。

AMFlow 2.0 采用一套经验配置 [5]：L 是圈数，假定的前导 Laurent 幂为 $n_{\min} = -2L$；它所谓的“阶”是从 $n_{\min}$ 向上计的重建深度 K，不是绝对最高幂。对十进制目标位数 g，

$$N_{sample} = \left\lceil \frac{5}{2}K + 2L \right\rceil, \tag{45}$$

$$\alpha_\epsilon = \frac{L}{2} + \frac{g}{K+1}, \tag{46}$$

$$\epsilon_0 \simeq 10^{-\alpha_\epsilon}, \qquad \epsilon_j = \epsilon_0 \left(1 + \frac{j}{100}\right), \quad j = 1, \ldots, N_{sample}, \tag{47}$$

$$p_0 = \max\left(\lceil (N_{sample} + 2L)\alpha_\epsilon \rceil, 30\right), \tag{48}$$

$$p_{work} = \lceil 2(1 + p_{extra})p_0 \rceil . \tag{49}$$

相应的求解器设置为 $X_{\text{order}} = 4p_0$ 和 $X_{\text{extra}} = \lceil \max(50, p_0/5) \rceil$。百分之一的间距、采样点有理化、加倍工作精度以及这两个展开阶数，都是 AMFlow 2.0 的实现选择，而不是式 (43) 和式 (44) 的推论。FlintNDE 把自动选择实现为 $\max(200, \lceil 2(1 + p_{extra})p_0 \rceil)$。200 位下限是 FlintNDE 自己的缺省值，不在所引 AMFlow 公式之列；显式给出 `working_precision_digits` 会覆盖自动规划。

FlintNDE 的接口不应要求圈数：L 是 AMFlow 针对费曼积分极点界所使用的量，不是一般矩阵微分方程的内禀参数。有限次数值调节参数求解也无法证明任意 Python 可调用对象的 Laurent 支撑。因此当前接口要求给出整数 $n_{\min}$，同时要求一份严格的上游证书：状态为 certified，记录的整数与 $n_{\min}$ 一致，方法指向符号的边界与 DE 分析。缺少证书、自动给出的前导幂或整数不一致，都会在任何 NDE 求解之前失败。MadStree 由其物理边界和 dlog DE 自动产生这份证书；FlintNDE 始终保持与这些物理数据结构无关。

显式的 `sample_points` 序列是一个有序的生产候选池。它可以比当前方阵插值所需更多：第一轮只取所需前缀，验证失败的一轮再取后面的候选点，而不重算此前的求解。可选参数 `initial_internal_maximum_power` 固定第一个方阵系统包含的最高幂。候选池一旦用完，重建不会生成用户批准范围之外的调节参数值，而是返回当前最好的系数，并置 `precision_target_met=False`，原因记为 `candidate_pool_exhausted`。为了给整个运行（含检查）设定上界，用户还必须另外提供一份与之不相交的 `validation_points` 序列。

另一种做法是用 `sample_angle_range=(theta_min,theta_max)` 为自动复数采样选定一个开放的弧度区间。实现至多取三条等距的内部射线，并在其上轮换由精度决定的半径。不使用边界射线，也没有用户的半径上限：模长仍取 $\epsilon_j = \epsilon_0(1 + j\delta)$，$\epsilon_0$ 由同一套受 AMFlow 启发的精度公式确定。自动验证采样保持相应角度，只缩放半径。

记已认证的全局前导幂为 $n_{\min}$，用户请求的绝对最高幂为 m，并定义

$$P = \max(0, -n_{\min}), \qquad K = m - n_{\min}. \tag{50}$$

自动生产规划在作透明替换 $2L \to P$、$L/2 \to P/4$ 后沿用 AMFlow 2.0 公式：

$$N_{fit} = \max\left(\left\lceil \frac{5}{2}K + P \right\rceil, K + 1 + \texttt{fit_extra_order}\right), \tag{51}$$

$$\alpha_\epsilon = \frac{P}{4} + \frac{g}{K+1}, \qquad \epsilon_0 \simeq 10^{-\alpha_\epsilon}, \tag{52}$$

$$p_0 = \max\left(\lceil (N_{fit} + P)\alpha_\epsilon \rceil, 30\right), \qquad p_{work} = \lceil 2(1 + p_{extra})p_0 \rceil . \tag{53}$$

缺省 fit_extra_order=2 时，第二个分支为 $K+3$。全部 N_{fit} 个生产采样用于从 $n_{\min}$ 到内部幂 $n_{\min}+N_{\mathrm{fit}}-1$ 的方阵插值，只返回请求幂 m 以内的系数。于是 m 以上的幂次像 AMFlow 的构造那样吸收截断效应；多出的生产点既不被丢弃，也不转成最小二乘拟合。至少 N_{check} 个不参与该插值的额外点用来验证保留的系数。所有分量的系数由同一个方阵 Acb 插值确定，并在这些独立点上检验。报告的结果包含上游证书、采用的前导幂、全部实际生效的参数、内部与返回的幂次范围、系数向量以及独立残差。

实现最初强制 m 以上至少有 fit_extra_order=2 个幂次。若独立残差过大，每一轮增加 fit_order_increment=2 个生产点和相应的幂次，只求解这些新点，此前的生产值和单独缓存的验证值继续复用。缺省的 fit_max_rounds=3 之后，就在不变的容差下拒绝。这个增量式的事后检验补充了上面的规划估计。

例如，上游证书给出 $n_{\min}=-4$，用户请求 $m=2$、$g=30$，则 $P=4$、$K=6$，并且

$$N_{\mathrm{fit}}=19,\qquad \alpha_\epsilon=\frac{37}{7}\simeq 5.285714,\qquad \epsilon_0\simeq 5.18\times 10^{-6}. \tag{54}$$

插值在内部覆盖 $-4,\dots,14$，但只返回 $-4,\dots,2$。同样的缺省值给出 $p_0=122$、$p_{work}=244$ 位十进制精度，主输运阶数 $4p_0=488$，附加输运阶数 $\lceil\max(50,p_0/5)\rceil=50$。在缺省的两个验证点下，这一阶段执行 19 次生产求解和 2 次独立验证求解。

6.5.1 幂级数解接口

已实现的高层命令是

```
series_result = reconstruct_series_solution(
    DEmatrix=DEmatrix,
    boundary=boundary,
    path=path,
    maximum_power=2,
    leading_power=-1,
    leading_power_certificate={
        "status": "certified",
        "leading_power": -1,
        "method": "symbolic-boundary-and-DE",
    },
)
```

它在每个调节参数采样处独立调用同一个基本 NDE 求解器。所有算法控制量都是可选的关键字覆盖项：

```
series_result = reconstruct_series_solution(
    DEmatrix=DEmatrix,
    boundary=boundary,
    path=path,
    maximum_power=2,                  # required target
```

```
    leading_power=-1,                # required certified support
    leading_power_certificate={
        "status": "certified",
        "leading_power": -1,
        "method": "symbolic-boundary-and-DE",
    },
    series_parameter="ep",
    goal_digits=30,
    sample_points="automatic",
    sample_count="automatic",
    base_sample="automatic",
    sample_spacing=0.01,
    working_precision_digits="automatic",
    extra_working_precision=0.0,
    transport_order="automatic",
    transport_extra_order="automatic",
    transport_sample_count="automatic",
    transport_extra_sample_count="automatic",
    radius_fraction=0.60,
    guard_bits=32,
    validation_sample_count=2,
    validation_points="automatic",
    validation_scale=0.5,
    validation_tolerance="automatic",
    maximum_samples=100,
    rationalize_sample_points=True,
    fit_extra_order=2,
    initial_internal_maximum_power="automatic",
    fit_order_increment=2,
    fit_max_rounds=3,
    parallel_task_count=12,
    output_layout=None,
    result_name="series_reconstruction",
)
```

这里 `transport_order` 是底层 NDE 求解的主局部级数阶数，`transport_extra_order` 是其精化检验所用的附加阶数，自动取值分别为 $4p_0$ 和 $\lceil \max(50, p_0/5) \rceil$。字面值 `"automatic"` 只选择文档记录的生产控制量；用户给出的值只替换该项决定。没有任何公开对象以 AMFlow 命名：引用只记录缺省经验公式的来源，而程序包的功能名为幂级数解重建。

前三个输入中的每一个都可以固定，也可以依赖调节参数，签名分别是 `DEmatrix(ep)`、

`boundary(ep)` 和 `path(ep, system)`。启用有理化后，生成的实采样以 FLINT `fmpq` 值传给这些工厂函数，因此依赖采样的精确有理矩阵仍能使用精确奇点门禁和 Frobenius 门禁。固定系统是 `AnalyticMatrixSystem` 或 `RationalMatrixSystem`；固定的普通边界是 Acb 列矩阵或标量列表；固定路径是直接的 `list[acb]`。奇点起点则使用第 6.6.4 节的精确 $\{a, b, C\}$ 协议；每个调节参数采样在输运开始前都复用同一套局部约化、指标和对数相容性检验。

6.6 完整公开接口与参数参考

用户环境只需要 Python 3.10 或更新版本，以及 `python-flint>=0.6`。发布后可以从软件包索引安装

```
python -m pip install FlintNDE
```

或者用下载的 wheel 安装

```
python -m pip install ./flintnde-1.0-py3-none-any.whl
```

下载的源码检出目录可以常规安装

```
python -m pip install path/to/FlintNDE
```

开发时也可以改用可编辑模式

```
python -m pip install -e path/to/FlintNDE
```

无论 wheel 还是源码安装，用户都不必手工构建程序包；`pip` 会自动安装声明的 `python-flint` 依赖。全部公开名字都从 `flintnde` 导入。精确标量接受整数、FLINT `fmpq`、有理或十进制字符串、`"a+b*I"` 这类字符串、二元组 `(real, imag)`，以及含 `"real"` 和 `"imag"` 的字典。公开的 `gaussian_rational` 转换器返回 $\mathbb{Q}(i)$ 中的 `GaussianRational`。Python 的 `float`、`complex`、`arb` 和 `acb` 值被精确路线拒绝；它们属于显式数值系统，并且必须声明可靠的输入精度。推荐的 `build_frobenius_manifest` 调度器遵循两条规则：

- `RegularSingularSystem` 选择精确分类，精确失败直接返回；
- `NumericalRegularSingularSystem` 对显式以浮点给出的数据选择数值分类。

6.6.1 归调用方所有的输出布局

程序包不往自己的安装树里写文件，也不从进程工作目录推断输出位置。每个调用脚本都可以初始化一个稳定的同级布局：

```
layout = initialize_output_layout(__file__, run_name=None)
summary_path = layout.write_json(
    "summary", "result_summary.json", summary,
)
```

安装一次之后，脚本可以放在任意目录 A 中导入 `flintnde`，不需要复制程序包源码。传入该脚本的 `__file__` 后，受管理的输出落在 `A/results/<run_name>/` 下。JSON、TOML 或其它配置文件可以和脚本放在一起并由脚本读取，但当前版本不提供只靠配置文件的命令行运行器：调用公开 API 仍然需要 Python 调用方。

缺省运行名是调用脚本的文件名去掉扩展名。生成的目录树为

```
calling_script.py
results/<run_name>/
    configuration/output_layout.json
    singularities/
    frobenius/
    transport/
    regularization/
    summary/
```

只有 configuration 目录及其布局清单会预先建立，其余类别在第一次使用时出现。精确的有限/无穷远清单放在 `singularities`；Frobenius 分类和幂–对数元数据放在 `frobenius`；路径、段和连接数据放在 `transport`；采样规划和 Laurent 系数放在 `regularization`；最终标量和检查放在 `summary`。

表 2: 归调用方所有的输出参数。

接口或参数	含义与约束
`initialize_output_layout`	参数为 `caller_script` 和仅限关键字的 `run_name=None`。前者是已存在的脚本路径，通常就是 `__file__`；后者缺省取它的主文件名，若给出，必须是单个非空路径分量。返回根在 `<caller>/results/<run_name>/` 的 `OutputLayout`。
`OutputLayout.directory(category)`	`category` 只能是 `configuration`、`singularities`、`frobenius`、`transport`、`regularization` 或 `summary` 之一。目录按需建立。
`OutputLayout.file(category, filename)`	`filename` 必须是单个非空相对路径分量，不能是绝对路径或嵌套路径。方法建立类别目录并返回文件路径，但不创建文件。
`OutputLayout.write_json`	参数为 `category`、`filename` 和可 JSON 序列化的 `payload`。写入缩进稳定的 UTF-8 JSON，并返回写入路径。
`OutputLayout` 字段	`caller_script`、`run_name`、`results_root` 和 `run_root` 是解析后的只读路径/元数据。用户通常从 `initialize_output_layout` 取得该对象，而不是直接构造。

6.6.2 精确 Gaussian 有理奇点与统一调度

多项式系数一律从常数项开始、按幂次递增排列。下面的输入同时含非实系数和非实极点：

```
layout = initialize_output_layout(__file__, run_name="point_01")
system = RationalMatrixSystem(
    (
        (rational_function([0, "1+I"], ["-2-3*I", 1]), 0),
        (0, rational_function(
```

```
            {"real": "1/3", "imag": "-2/5"}, [1, 0, 1]
        )),
    ),
    variable_name="s", name="example-system",
)
inventory = analyze_singularities(system, output_layout=layout)
path = build_adaptive_path(
    system,
    NamedPoint("left_boundary", 2),
    NamedPoint("right_match", 3),
    path_name="left_to_right",
    max_step_over_radius=0.45,
    output_layout=layout,
)
local = prepare_local_expansion(
    system, NamedPoint("left_boundary", 2),
    order=32, radius_fraction=0.60, sample_count=96,
    output_layout=layout,
)
snapshots, segment_reports, elapsed = transport_path(
    system, column_vector([1, 0]), path,
    order=32, sample_count=96, radius_fraction=0.60,
)
```

整数、FLINT `fmpq`、有理或十进制字符串、`"a+b*I"`、实/虚二元组以及实/虚字典都是精确系数输入。自动分母发现拒绝 Python 的 `float`、`complex`、`arb` 和 Acb 值。有理项之和应当用普通的 `+` 构造；得到的完整矩阵元会先约化，再收集因子。黑箱 `AnalyticMatrixSystem` 无法从有限次采样暴露全部极点，必须继续显式声明其奇点。由于 `rational_function((c0,c1))` 的顶层元组表示多项式系数，单个实/虚二元组要包进单元素系数列表，或先用 `gaussian_rational` 转换；这样就消除了唯一的语法歧义。落在 $\mathbb{Q}(i)$ 中的根保留精确位置，可以直接交给精确 Frobenius 构造器。更一般的代数根仍在清单中作为 Acb 球分离，但一般代数数域上的精确 Frobenius 分析未实现。同样，不在 $\mathbb{Q}(i)$ 上完全分裂的指标多项式会被直接拒绝，而不是回退到阈值判定。

表 3: 精确奇点、局部调度与命名路径参数。

接口或参数	含义与约束
`rational_function`	参数为 `numerator` 和 `denominator=(1,)`。标量被提升为单个系数；列表或元组按幂次递增排列。返回精确的已约化 `RationalFunction`。

接口或参数	含义与约束
gaussian_rational	参数为 value=0 和可选的 imag=None。它校验并把上面列出的精确标量形式转换为 GaussianRational；浮点输入抛出 TypeError。
GaussianRational	只读字段 real 和 imag 是 FLINT fmpq 值。提供精确算术、共轭、Acb 转换以及 $\mathbb{Q}(i)$ 内的认证平方根；该域之外的根直接拒绝。
RationalFunction	构造字段为系数元组 numerator 和 denominator=(1,)。加法精确合并同类项。方法 evaluate(point)、exact_polynomials()、inverted_variable() 和 pole_order_at_infinity() 给出约化求值与无穷远变换。
RationalMatrixSystem	构造参数为方阵嵌套的 entries、variable_name="s" 和 name。元素可以是 RationalFunction、精确常数，或含 numerator 与可选 denominator 的字典。强制转换后字段不可变。
singularity_inventory	仅限关键字的 root_tolerance=1e-30。它对每个约化分母做精确分解，分离全部有限复根，并单独判定无穷远点。返回 SingularityInventory。
analyze_singularities	参数为 system；仅限关键字的 root_tolerance=1e-30、output_layout=None 和 filename="singularity_inventory.json"。给出布局时，全部位置、极点阶数、因子和见证矩阵元都保存在 singularities 下。
RationalMatrixSystem.inverted	关键字 variable_name="sinv"；返回式 (17) 的精确矩阵。
NamedPoint	构造字段为 name 和 value。有限值接受通常的 Acb 兼容标量形式；无穷远只接受字面字符串 "inf"。
classify_point	参数为一个 NamedPoint 和 SingularityInventory。返回含类型、极点阶数、匹配奇点标识和收敛半径的 PointClassification。
prepare_local_expansion	参数为 system、endpoint；仅限关键字的 order、radius_fraction=0.60、radius=None、sample_count=None 和 output_layout=None。它选择 ordinary_cauchy_dft、regular_singular_power_log、fuchsian_reduced_power_log、exponential_power_log 或只在起点的 formal_exponential_asymptotic。无界的普通圆盘必须显式给出 radius；形式基不报告收敛半径；未认证的高阶极点抛出 LocalReductionError。
LocalExpansion	只读字段为 classification、working_variable、method、convergence_radius、可选的 matrix_coefficients、可选的 power_log_basis 以及 manifest。
attempt_fuchsian_reduction	参数为精确的 RationalMatrixSystem 和一个 $\mathbb{Q}(i)$ 点。它迭代式 (31) 的精确 Lee–Moser 平衡，返回有序投影、原始与约化后的极点阶数、完整极点阶数历史、状态、原因，以及经过精确往返认证的变换对象。

接口或参数	含义与约束
build_local_solution_basis	参数为精确系统、点和正的级数阶数。它调度正则幂–对数、剪切后幂–对数、认证的指数乘幂乘对数，或简单谱秩一形式递推，并返回原始积分基中的 LocalSolutionBasis。形式路线的 continuation_ready 为假；evaluation_report 给出固定阶数、五阶块比与精化，以及辅助的最小项诊断。不支持的 form/Stokes 问题抛出 LocalReductionError。
five_term_tail_diagnostic	参数为算到 $N+5$ 阶的标量项、求值阶数 $N \geq 4$，以及可选的正阈值。返回实现式 (38) 的 FiveTermTailDiagnostic；取等号不算通过。
build_adaptive_path	参数为 system、start 和 target；仅限关键字的 detour_points=()、path_name=None、max_step_over_radius=0.45、path_tolerance=1e-24、formal_asymptotic_order=None、formal_minimum_order_factor=3.0 和 output_layout=None。正的形式阶数启用根间隙形式起点估计；因子必须大于一。返回 AdaptivePath，它是 list[acb]，可原样作为 transport_path 的 path 参数。每个报告的步都满足 $\lvert\Delta z\rvert/R \leq$ max_step_over_radius。可衔接的内部极点给出警告并接受局部桥接；只在起点的形式点若位于内部或作为目标则被拒绝，因为缺少 Stokes 数据。有限的普通 detour_points 仍是可选项。
AdaptivePath	除常规列表操作外，只读诊断包括 step_reports、internal_singularities、start_classification、target_classification、working_variable、infinity_transformation、formal_asymptotic_match_estimate 和 max_step_over_convergence_radius。有序报告含步长、控制半径、半径所有者、比值、方法和奇点标识。进入奇点的步总是把该奇点记为半径所有者。
build_adaptive_path_plan	诊断接口，使用同样的命名端点、步长比、路径容差、local_analysis_order=4 和输出布局。它返回元数据而不是数值路径参数，并对每个高阶极点检查点做预检。
PathPlan	只读字段包括两个变量名、无穷远变换标志、端点分类、全部内部奇点、生成的命名点、分段方法、步长/半径比、消息以及 continuation_ready。正则奇点，以及具备认证剪切或收敛指数局部基的高阶极点，是可衔接的。形式渐近点仅在作为起点时可衔接；位于内部或目标时该标志为假，并记录缺失的 Stokes 连接。
SingularityRecord	保存标识、Acb 位置、可选的精确线性根、极点阶数、分类、精确约化分母因子和从零开始计数的见证矩阵元。
SingularityInventory	保存全部有限记录、无穷远记录、系统名、变量名和 JSON 序列化器。

接口或参数	含义与约束
plan_transport_path	参数为 system、start 和有序的 points 列表；仅限关键字的 radius_fraction=0.60、singularity_mode="avoid"、singularity_jump_threshold=0.5、match_fraction=0.6、max_singularity_jumps=16 和 message_language="EN"。缺省的 avoid 模式不插入奇点节点，折线上有极点时抛出 SingularPathError；显式的 singularity_jump 模式允许恰好有一个中间用户点落在简单极点上，把它从普通节点链中移除，并要求穿越线段共线。奇点附近 radius_fraction 会被压小，并报告实际生效值。返回 PlannedPath。
direct_user_point_path	参数为 system、start 和 points；仅限关键字的 message_language="EN"。用户点原样成为输运节点，不插入也不删除任何点。它只做极点安全检查，折线穿过极点或端点落在极点上时拒绝。
transport_planned_path	参数为 system、Acb 的 initial_vector 列和 PlannedPath；仅限关键字的 order、sample_count=None、radius_fraction=0.60、target_relative_error=None 和 collect_patches=False。奇点跳越段使用其极点的缓存局部基，普通段使用 Taylor 递推，步内用户点用 Horner 代入求值。返回快照、分段报告、耗时秒数和步内采样结果摘要。
transport_planned_path_refined	相同输入，另加 primary_order、reference_order、可选的 reference_initial_vector=None 和 certification_mode="certified"。多段规划和含奇点跳越的规划会把 embedded 请求升级为独立的 certified 双链。用户可见的节点和稠密数值来自主链；参考链只校验精度。
plan_singular_target_match	参数为精确的 RationalMatrixSystem、普通 initial_point 和奇点 target_point；仅限关键字的 radius_fraction="1/4"。当前点若已在保守局部圆盘内就直接复用，否则沿入射方向插入一个隐藏的普通匹配点。它只做规划；两条链都必须先输运到公共匹配点。
evaluate_singular_target	参数为精确系统、同一分支上的普通 initial_point、输运后的 initial_vector 和奇点 target_point；仅限关键字的 order=48、target_relative_error=1e-25 和 sample_exponents=(8,12,16,20,24)。它通过认证的局部幂–对数基求值，不做数值绕行：有限分量返回最后一个局部采样，真实极点或零阶对数发散分量返回文本 Infinity。SingularTargetResult 给出逐分量分类，以及采样量级、指数和局部基阶数。
planned_path_to_json; planned_path_from_json	序列化使用仅限关键字的 digits=80（下限 20），并恢复 PlannedPath。执行器按当前系统的极点校验记录，不得重新运行规划器；给出 system 时，极点编号和坐标必须一致。

接口或参数	含义与约束
adaptive_path_to_json; adaptive_path_from_json	AdaptivePath 遵循同样的两阶段约定：节点和局部基桥接以 digits=80 保存；恢复时要求同一个 RationalMatrixSystem，并重新校验奇点清单而不是重新规划。
失败类别	SingularPathError 报告规划折线上的极点或不支持的奇点路线；SingularityJumpError 报告跳越模式下局部基构造或穿越失败；AdaptivePathSingularityError 报告自适应构造无法路由的内部奇点。

6.6.3 普通点工作流

最小的普通点计算是

```
from flint import acb, acb_mat
from flintnde import *

# The import has already selected the 200-digit default.
system = AnalyticMatrixSystem(
    matrix_function=lambda z: acb_mat([[0, 1], [-1/(1-z), 0]]),
    dimension=2, singularities=(acb(1),), name="example",
)
path = build_straight_path(system, acb("0.1"), acb("0.7"))
result = transport_path_refined(
    system, column_vector([1, 0]), path,
    primary_order=24, reference_order=30,
)
```

回调必须返回声明维数的 acb_mat。凡是可能限制 Cauchy 圆盘的奇点都必须列出。空列表只是关闭自动半径与路径控制，并不意味着断言矩阵是整函数。

表 4: 精度、矩阵系统与输运参数。

接口或参数	含义与约束
configure_working_precision	参数为 decimal_digits 和 guard_bits=32。把进程级 FLINT 上下文设为 $\lceil \texttt{decimal_digits} \log_2 10 \rceil + \texttt{guard_bits}$ 位，并返回该位数。十进制位数必须为正，保护位必须非负。
DEFAULT_WORKING_PRECISION_DIGITS; DEFAULT_PARALLEL_TASK_COUNT; FLINTNDE_CITATION_NOTICE	模块常量：第一个是导入时的缺省精度，第二个是批处理和重建入口的缺省工作进程上限，第三个是每个进程导入时打印一次的引用提示。

接口或参数	含义与约束
`AnalyticMatrixSystem`	构造参数为 `matrix_function`、`dimension`、`singularities=()` 和 `name`。回调 `matrix_function(z)` 返回系数矩阵；`dimension` 是方阵维数；`singularities` 提供用于半径控制的 Acb 位置；`name` 用于标注错误和报告。
`evaluate(point)`	调用回调并检查声明的维数。
`nearest_singularity_distance`	其 `point` 参数是 Acb 值。返回声明奇点中的最小距离；未提供奇点时抛出错误。
`PartialFractionSystem`	构造字段为常数矩阵 `constant`、并行的 `residues` 与 `poles` 元组、以及可选的 `polynomial_coefficients=()`；它表示 $A(z) = P(z) + \sum_j R_j/(z - p_j)$，$P$ 按次数逐项存储。有限极点必须简单且互异；显式构造该系统即接受该前提。只读属性为 `dimension`、`singularities` 和 `polynomial_degree`。
`taylor_matrix_coefficients`	参数为 `center`、`order`，仅限关键字的 `radius` 和 `sample_count=None`。它重建 $A_0, \ldots, A_{\texttt{order}-1}$。`radius` 非零，且必须落在解析域内。缺省采样数为 $\max(32, 2\,\texttt{order})$，用户给出的采样数必须大于 `order`。
`build_straight_path`	参数为 `system`、`start`、`target`，以及仅限关键字的 `step_fraction=0.2`。它构造直线 Acb 路径，每步至多为 `step_fraction` 乘最近奇点距离，该分数严格位于 $(0, 1)$。必须避开极点的路径应改为给出复数绕行点。
`transport_path`	参数为 `system`、`initial_vector`、`path`，以及仅限关键字的 `order`、`sample_count=None`、`radius_fraction=0.6` 和 `target_relative_error=None`。它沿至少两个路径点输运一个 Acb 列。若起点是精确奇点，`initial_vector` 改为第 6.6.4 节的 `FrobeniusBoundary`；参数名为兼容 API 而保留。`order` 是 Taylor 阶数；`sample_count` 遵循上面的 Cauchy 规则；`radius_fraction` $\in (0, 1)$。`build_adaptive_path` 返回的 `list[acb]` 可直接使用。路径可能含奇点局部桥接时，要传入原始的 `RationalMatrixSystem`；普通列表仍可用 `AnalyticMatrixSystem`。通常 `radius_fraction` 应大于 `max_step_over_radius`，因为每个普通步都必须留在自己的采样圆内。桥接报告记录幂–对数阶数、最大对数次数、分支约定、两个步长/半径比和耗时。在形式起点，可选精度目标与 δ_5 比较，并且总会检查 $r_5 < 1$；失败会警告，但不会撤掉结果。函数返回 `(snapshots, segment_reports, elapsed_seconds)`。

接口或参数	含义与约束
`transport_path_refined`	除 `system`、`initial_vector` 和 `path` 外，仅限关键字的参数为 `primary_order`、`reference_order`、`primary_sample_count=None`、`reference_sample_count=None`、`radius_fraction=0.6` 和 `target_relative_error=None`。它运行相互独立的主链与参考链，参考阶数必须高于主阶数。返回的字典含两条链、计时、分段报告、最终无穷范数相对差和 `target_relative_error_met`。目标未达标会给出警告，但两条链都会保留。
`transport_frobenius_boundaries_refined`	参数为精确的 `RationalMatrixSystem`、非空的 `FrobeniusBoundary` 列表和一条公共路径，随后是 `primary_order`、`reference_order`、`radius_fraction=0.6` 和 `target_relative_error=None`。在每个阶数内，它只构造一次局部基，把独立给出的边界排成矩阵列，并复用每段普通 Taylor 矩阵。返回字典含矩阵快照、共享分段报告、逐列边界报告、`relative_differences_inf` 和逐列的 `target_relative_error_met`。当前批量路线要求奇点发射之后的每一步都是 `ordinary_taylor`，并拒绝坐标 `save` 标记。这只是批量优化的限制：普通的 `transport_path` 和 `transport_path_refined` 接口会把直接的正则奇点起点或目标保存为可复用的 `frobenius_boundary` 数据，其中含已校验的 (a, b, C) 项。需要保存普通点或奇点边界常数时，调用方分别调用 `transport_path_refined`。
`column_vector(values)`	把 Acb 或精确实标量列表转换为 Acb 列矩阵。
`acb_midpoint_matrix(matrix)`	由高精度中点重建矩阵；这能限制区间包裹，但丢弃包围半径。
`vector_norm_inf(vector)`	返回列向量中模最大的元素。
`relative_difference_inf`	参数为同维的 `primary` 与 `reference` 列。同维时返回 $\|\texttt{reference} - \texttt{primary}\|_\infty / \|\texttt{reference}\|_\infty$；若参考范数的球包含零则拒绝。

6.6.4 正则奇点工作流

对精确的 $\mathbb{Q}(i)$ 数据，推荐的工作流是

```
system = RegularSingularSystem(
    residue_exact=[["I", "0"], ["0", "1+I"]],
    regular_coefficients_exact=([["0", "0"], ["1", "0"]],),
    name="resonant-example",
)
manifest = build_frobenius_manifest(system)
basis = build_power_log_basis(system, manifest, series_order=20)
fundamental_matrix = basis.evaluate(acb("0.01"))
```

元组 `regular_coefficients_exact` 按 $(A_0, A_1, \ldots)$ 排列；缺少的高阶系数取零。清单包含根及其重数、半单性数据、每个共振门禁，以及这些门禁实际启用的最大对数次数。

对数值数据，使用

```
system = NumericalRegularSingularSystem(
    residue, regular_coefficients, name,
    input_precision_digits=80,
)
options = NumericalFrobeniusOptions(relative_zero_tolerance=None)
manifest = build_frobenius_manifest(system, options)
```

警告是无条件给出的，即使提供了用户阈值也一样。`numeric_diagnostics` 块记录 `matrix_scale`、两个截断阈值和置零个数；被置零的最大量与保留的最小量分别给出绝对值以及相对矩阵尺度的值。

奇点起点的边界数据。 奇点处一般没有 Y 的有限值。因此边界输入给出一项或多项前导幂–对数因子，

$$\{a_i, b_i, C_i\} \quad \longleftrightarrow \quad z^{a_i}(\log z)^{b_i} C_i, \tag{55}$$

其中 C_i 是全部主积分的前导系数向量，含其零分量。有限起点 s_0 处 $z = s - s_0$，无穷远处 $z = \mathtt{sinv} = 1/s$。因此 a_i 始终是局部系统实际所用局部变量中的指数。

可执行的输入是

```
boundary = frobenius_boundary([
    {"a": "0", "b": 1, "C": [1, 0]},
    {"a": "0", "b": 0, "C": [2, 0]},
])


path = build_adaptive_path(
    DEmatrix,
    NamedPoint("start", 0),
    NamedPoint("target", 1),
```

```
)
snapshots, reports, elapsed = transport_path(
    DEmatrix, boundary, path, order=24,
)
```

每条记录也可以写成元组 `(a, b, C)`。`a` 和 `C` 必须是精确的 $\mathbb{Q}(i)$ 输入；Python 的浮点、复数、Arb 和 Acb 值都被拒绝。对数次数 `b` 必须是非负 Python 整数，非零向量 `C` 的维数必须等于微分方程的维数。普通起点仍要求给出有限 Acb 列。在奇点起点给出该列，或在普通起点给出 Frobenius 记录，都会在第一个输运段之前失败。

对半单留数，门禁要求 $b_i = 0$ 且 $C_i \in \ker(R - a_i\mathbf{1})$。对单根 Jordan 扇区 $R = a\mathbf{1} + N$，次数为 b 的记录通过

$$N^b c = b!C, \qquad N^{b+1}c = 0 \tag{56}$$

确定规范基常数 c。程序包用 RREF 求解这个精确线性系统，自由变量取零；随后由 $z^a \exp(N \log z)c$ 生成全部较低次对数幂，用户不必在输入里重复这些系数。多条记录线性叠加。数值求值之前，程序会检查精确指标归属、本征空间或 Jordan 链相容性、最大对数次数以及合成解是否非零。更高阶的共振对数仍是已校验幂–对数基的性质，会被自动生成。

对可衔接的不规则扇区，可复用的记录是

$$\{\phi_i, a_i, b_i, C_i\} \quad \longleftrightarrow \quad \exp\big(\phi_i(z)\big) z^{a_i} (\log z)^{b_i} C_i, \qquad \phi_i(z) = \sum_{p<0} \phi_{i,p} z^p. \tag{57}$$

每个 p 都是负的 Python 整数，每个 $\phi_{i,p}$ 都在 $\mathbb{Q}(i)$ 中精确。可执行形式为

```
boundary = exponential_boundary([{
    "phi": [
        {"power": -2, "coefficient": "3/2"},
        {"power": -1, "coefficient": "-1"},
    ],
    "a": "2/3", "b": 0, "C": [1, 0],
}])
```

重复的幂次会合并，精确为零的系数会删除。空的 `phi` 列表表示 $\phi = 0$、$\exp(\phi) = 1$，这样零特征扇区与同一高阶极点系统的其它扇区使用同一种可复用格式。该记录不允许用户另选指数：Flint-NDE 从高阶 Laurent 矩阵导出每个特征并要求精确相等。因此指数输入必须提供 $\{\phi, a, b, C\}$，保存的输出记录同样的完整 schema。

对经 Moser 约化的高阶极点起点，同一记录按原始积分基解释。程序包把精确的约化基幂–对数射流乘上完整的 Laurent 规范，并在该基中求解较早幂次、更高次对数和请求的 C_i 约束；它不是只变换孤立的前导向量。对指数起点，C_i 的精确常数基变换确定扇区，而要求的输入 ϕ 会与式 (34) 的指数对照。一个项不得混合不同的指数特征。两种情况下，把物理边界拆成多条记录即表示所需的线性叠加。

校验之后，局部基在自适应路径的第一个普通匹配点求值，普通输运从该值继续。第一个分段报告会写明实际的正则、Moser 约化或指数初始化路线；它记录原始项、约化清单、规范基常数、匹配点、工作变量，以及主 Acb 对数/幂分支。`reconstruct_series_solution` 的每个生

产和验证采样都独立使用同一协议。数值 Frobenius 清单以及既有的混合根亏损精确情形仍保持拒绝，因为二者目前都不能为这种边界转换提供经过认证的规范链基。

表 5: 奇点局部基与幂–对数参数。

接口或参数	含义与约束
`RegularSingularSystem`	构造参数为 `residue_exact`、`regular_coefficients_exact` 和 `name`。留数是 R；`regular_coefficients_exact` 是 A_m 矩阵组成的元组；`name` 是清单标识。所有矩阵同维，且元素都是精确的 $\mathbb{Q}(i)$。
`frobenius_boundary`	其 `records` 参数是非空可迭代对象，元素是字段恰为 `a`、`b`、`C` 的精确映射，或 `(a,b,C)` 元组。它立即校验记录 schema 和精确标量字段，返回冻结的 `FrobeniusBoundary`；依赖 DE 的指标与对数检验在输运时进行。
`exponential_boundary`	其 `records` 参数是非空可迭代对象，元素是含 `phi`、`a`、`b`、`C` 字段的精确映射。`phi` 是可能为空的列表，元素为负整数 `power` 与精确 `coefficient` 组成的对。返回 `ExponentialBoundary`；具体局部基稍后会按推断出的指数扇区校验完整特征。
`FrobeniusBoundaryTerm`	构造字段为 `a`、`b` 和 `C`。它们保存式 (55) 中一项精确的最高次对数前导项。直接构造与工厂函数执行同样的校验。
`FrobeniusBoundary`	其 `terms` 字段是非空的 `FrobeniusBoundaryTerm` 元组，各元素向量维数相同。用户通常通过 `frobenius_boundary` 构造它。
`build_frobenius_manifest`	参数为 `system` 和可选的 `options=None`。`RegularSingularSystem` 一律选择精确门禁并拒绝数值选项；精确失败直接返回。`NumericalRegularSingularSystem` 选择数值门禁，可以接收 `NumericalFrobeniusOptions`。
`build_exact_frobenius_manifest`	唯一参数是 `system`。它是显式的底层精确入口，在原始维数上做 $\mathbb{Q}(i)$ 指标、Jordan、投影和共振判定。指标多项式必须在 $\mathbb{Q}(i)$ 上完全分裂；不支持的混合根亏损结构直接拒绝。
`exact_data`; `acb_data`	`exact_data()` 不取参数；`acb_data(order)` 要求正的 `order`。它们返回已校验的精确矩阵或其 Acb 转换。正的 `order` 控制返回多少个正则系数，缺少的 A_m 用零矩阵补齐。
`NumericalRegularSingularSystem`	构造参数为 `residue`、`regular_coefficients`、`name` 和 `input_precision_digits=None`。它接受可转成 Acb 的浮点值，包括 `acb`、`arb`、复数值、实/虚二元组和十进制字符串。当可靠十进制精度无法从机器浮点或球推断时，最后一个参数负责记录它。
`acb_data`（数值系统）	不取参数，返回已校验的 Acb 留数以及给出的正则系数列表，不补高阶零系数。

接口或参数	含义与约束
NumericalFrobeniusOptions	构造参数为 precision_digits=None 和 relative_zero_tolerance=None。前者声明可靠的输入位数。Python binary64 输入即使给了更大的值，也按 15 位封顶。若输入以及系统/选项都没有声明精度，分类拒绝。缺省相对阈值为 $N10^{-p}$；以正十进制字符串给出的 relative_zero_tolerance 会覆盖它。
build_numerical_frobenius_manifest	参数为 system 和 options=None。它是显式的底层 Acb 中点、依赖阈值的入口。省略 options 时，仍要求由数值系统或其机器/球输入提供精度信息。不确定的情形一律拒绝。
build_power_log_basis	参数为 system、manifest，以及仅限关键字的 series_order。它复用精确或阈值门禁后的结构，算到正的 series_order 阶。系统名和清单状态必须一致。
PowerLogBasis.evaluate	其 point 参数是非零 Acb 值。它在该点计算基矩阵，使用由该点选定的 FLINT 对数分支。
PowerLogBasis.dimension、maximum_log_degree、manifest	只读结果字段。用户通过 build_power_log_basis 获得该对象；其私有的求值器构造参数不是用户选项。

6.6.5 幂级数解重建

公开入口是第 6.5 节所示的单个高层命令，以及供外部已采集采样使用的底层函数 fit_sampled_series；采样循环本身不是单独的公开类。最小的高层调用只需给出微分方程、边界、路径和期望的最高调节参数幂；前导幂和全部数值控制量在未覆盖时自动选取。

表 6: 幂级数解重建参数。

接口或参数	含义与约束
DEmatrix、boundary、path	必填输入，在每个调节参数值处传给同一个基本 NDE 求解器。它们可以是固定对象，也可以是签名为 DEmatrix(ep)、boundary(ep) 和 path(ep, system) 的工厂函数。路径采用与普通 NDE 调用相同的直接列表格式。边界是普通点列向量；路径从精确正则奇点开始时，则是第 6.6.4 节的 FrobeniusBoundary。边界工厂在每个调节参数采样处可以返回两种格式之一；格式不兼容时拒绝。
maximum_power	必填，返回给用户的绝对最高调节参数幂 m。没有公开的 series_range；最低幂单独给出。
leading_power	必填整数 $n_{\min}$，在任何数值 NDE 工作之前由符号边界条件和 DE 得到。字面值 "automatic" 被拒绝。
leading_power_certificate	必填映射，必须恰含 status="certified"、与 leading_power 相同的整数，以及非空的 method。FlintNDE 在任何采样求解之前校验该约定。
series_parameter="ep"	外层采样循环所重建参数的名称。
goal_digits=30	正的请求十进制精度 g，供自动采样、工作精度和验证规则使用。

接口或参数	含义与约束
`sample_points="automatic"`	自动时使用式 (53) 和有有理化的百分之一网格。显式有序序列是候选池：每一轮只取当前内部最高幂所需的前缀，复用此前数值，候选池用完后绝不再生成点。
`sample_angle_range="automatic"`	可选的开放弧度区间 $(\theta_{\min}, \theta_{\max})$，用于自动复数采样。至多使用三条等距内部射线，半径仍自动决定。该选项与显式 `sample_points` 互斥；自动模式保持正实轴网格。
`sample_count="automatic"`	自动的生产插值个数。给出显式 `sample_points` 时，显式个数必须等于候选池总长，而不是第一轮消费的个数。
`base_sample="automatic"`	生产尺度 ϵ_0；自动时取式 (53) 的 $10^{-\alpha_\epsilon}$。
`sample_spacing=0.01`	$\epsilon_j = \epsilon_0(1 + j\delta)$ 中的相对间距 δ。
`working_precision_digits="automatic"`	每次 NDE 求解的十进制精度；自动时取式 (53) 中的 p_{work}。
`extra_working_precision=0.0`	在 $p_{\mathrm{work}} = \lceil 2(1 + \texttt{extra_working_precision})p_0 \rceil$ 中按比例增加的安全余量；给出 `working_precision_digits` 时忽略。
`transport_order="automatic"`	每个底层 NDE 输运所使用的主局部级数阶数；自动时为 $4p_0$。
`transport_extra_order="automatic"`	精化求解所用的附加局部级数阶数；自动时为 $\lceil \max(50, p_0/5) \rceil$。
`transport_sample_count="automatic"`	主 NDE 链的 Cauchy–DFT 圆采样数；自动时用基本求解器规则 $\max(32, 2n)$。
`transport_extra_sample_count="automatic"`	参考链的 Cauchy–DFT 圆采样数，在更高阶数下使用同样的自动规则。
`radius_fraction=0.60`	用作每个普通点 Cauchy 半径的最近奇点距离分数。
`guard_bits=32`	把十进制工作精度换算到全局 FLINT 上下文后附加的二进制保护位。
`validation_sample_count=2`	不参与插值、留给事后残差检查的求解数 N_{check}。
`validation_points="automatic"`	可选的显式独立网格。这些值绝不进入拟合，并且必须与整个生产候选池不相交。同时给出显式的生产网格和验证网格，才能把每次调节参数求解限制在用户批准的范围内。
`validation_scale=0.5`	独立验证网格相对于生产网格的缩放。
`validation_tolerance="automatic"`	相对残差容差；自动时目标为 10^{-g}，同时尊重 Acb 包围半径，并报告实际生效的阈值。
`maximum_samples=100`	N_{fit} 的上限，与 AMFlow 的生产采样上限一致。验证求值单独报告。
`rationalize_sample_points=True`	在每次 NDE 求解之前，把自动生成的十进制尺度和间距转成精确有理的调节参数值。取 False 时改为把 Acb 采样传给工厂函数。
`fit_extra_order=2`	请求的最高幂之上，第一轮自动缓冲的最小幂次数。初始内部幂为自动时，该参数保持原有含义。
`initial_internal_maximum_power="automatic"`	可选的绝对第一轮内部最高幂 $q \geq m$。在已认证前导幂 $n_{\min}$ 下，第一轮恰好需要 $q - n_{\min} + 1$ 个生产候选；容量不足会在任何 NDE 求解之前失败。

接口或参数	含义与约束
`fit_order_increment=2`	每轮验证失败后追加消费的候选数。此前算出的生产和验证值继续缓存。
`fit_max_rounds=3`	拟合/验证轮数的上限。若存在方阵拟合但目标残差仍未达到，则返回当前系数且不做精度认证；诊断会区分候选耗尽、采样上限和轮数上限。
`parallel_task_count=12`	独立固定调节参数工作进程的最大数目。实际生效数为 `min(parallel_task_count, 采样数)`；排队的采样在工作进程空出时自动开始。固定的精确有理系统、普通边界向量和序列化的自适应路径在每个工作进程中恢复。任意的 `AnalyticMatrixSystem` 实例和局部闭包必须改为模块级工厂，或以并发数一运行。
`output_layout=None`	可选的归调用方所有的 `OutputLayout`。给出时，JSON 结果只写入其 `regularization` 类别下。
`result_name="series_reconstruction"`	JSON 文件名所用的单个路径安全词干。
`SeriesReconstructionResult`	提供 `powers`、系数列向量、生产与验证点/值、实际生效参数、诊断、`coefficient(power)` 和 `evaluate(ep)`。
`fit_sampled_series`	供外部已采集采样使用的底层拟合入口。仅限关键字的参数为 `sample_points` 与 `sample_values`，可选 `regularization_points=None`，必填 `maximum_power`、`leading_power=0`，可选 `validation_points()`、`validation_values()`、`validation_regularization_points=None`、`validation_tolerance=None` 和 `series_parameter="ep"`。缺省时采样点就是插值变量；给出显式正则点时，前者仍是采样标签，后者进入 Vandermonde 系统，从而可由成对的 $\pm\delta$ 标签重建实奇点两侧的侧向级数。标量采样提升为单列向量，全部生产点进入一次方阵求解，不使用最小二乘或伪逆；残差越界时拒绝。该入口不改变全局 FLINT 精度；调用方自行配置精度，并在该精度下生成采样。
失败类别	`ValueError` 拒绝缺失或不一致的前导幂证书；`SeriesValidationError` 拒绝未解决的 NDE 精化、插值或独立采样残差。

6.6.6 独立的 C^{++} 引擎

引擎消费与 Wolfram 批处理入口相同的序列化约定。单次输运使用 schema `madstree_flintnde_evaluate_v1`，固定调节参数批处理使用 `madstree_flintnde_ep_batch_v1`；级数控制 schema `madstree_flintnde_ep_series_control_v1` 会被识别，并以状态 `seriesControlNotImplemented` 在结构上拒绝，调节参数级数控制仍留在 Python 和 Wolfram 路线上。矩阵元是 `{alpha, beta, residue}` 形式的 dlog 字母，`alpha` 和 `beta` 用精确有理字符串；十进制字符串被拒绝。`beta=0` 的字母被跳过，`alpha=0` 被拒绝，同一极点上的留数会被合并。批处理文件采用流式解析：只预先读取文件头，每个工作进程解析并丢弃自己的任务；

`--workers` 缺省取批处理字段 `parallelTaskCount`。

输运只保留前缀模式。由于主链与参考链共享精度、只在阶数上不同，每条参考链级数的前 `primaryOrder`+1 个系数与主级数相同，因此每步一次不整齐的递推就能同时产出两条链；输出与两条独立链逐位一致，相对差认证和 `targetRelativeErrorMet` 仍照常执行。留数内核由 `--residues` 选择：`auto` 在留数密度不超过 0.25 时用稀疏内核，否则用稠密内核。三种内核逐位一致；`flintndeEngine` 元数据按请求和按段记录所请求的与实际生效的内核。

Wolfram 直接入口 `FlintNDEEngineEvaluateEpBatch` 绕过 Python 桥接。它的 `system` 只接受无多项式的 `PartialFractionSystem`；位于零的极点无法编码，返回 `EngineZeroPoleUnrepresentable`；非精确标量返回 `EngineInexactScalar`；非实 `start` 返回 `EngineRealStartRequired`。`WorkingPrecisionDigits` 缺省 200、下限 20；`ReferenceOrder` 必须高于 `PrimaryOrder`；`ParallelTaskCount` 缺省 12，实际生效数由任务数封顶。缺省的 `Engine` 可执行文件是版本根下 `engine/build/` 中的二进制；Windows 上 `EngineLibraryPath` 指向 `libflint.dll` 所在目录。引擎只执行 `pathPlanning=false`、`singularityMode="avoid"` 的用户点序列；奇点跳越、Frobenius 与指数边界以及保存点仍留在通用的 Python 和 Wolfram 路线上。

6.7 连接矩阵与边界子空间

设 Φ_a 和 Φ_b 是两个端点附近的局部基矩阵，U_γ 是沿路径 γ 的普通点输运，则连接矩阵为

$$C_{ba}^{(\gamma)} = \Phi_b^{-1} U_\gamma \Phi_a. \tag{58}$$

若 V_a 张成允许的初始边界子空间，L_b 湮灭允许的末态子空间，则非零的双端解满足

$$L_b C_{ba}^{(\gamma)} V_a a = 0. \tag{59}$$

这一形式同时覆盖主积分边界匹配和谱边值问题。

7 软件现状与展望

当前开发版只依赖 python-flint>=0.6 和 sympy>=1.12，包含通用的 Cauchy–DFT 普通点后端、精确或按精度阈值门禁的幂–对数基、精确的 $\{a, b, C\}$ 奇点边界初始化、精确的 Lee–Moser 投影平衡、经过认证的解耦指数扇区、只在起点的简单谱形式渐近及其固定阶五阶诊断、可由用户覆盖的调节参数重建、第 6.6.6 节的独立 C++ 引擎及其 Wolfram 直接入口、单元测试，以及二维边界例子。一个 16 维复精确测试还检验了 $\mathbb{Q}(i)(k)$ 上通用的 `RationalMatrixSystem`：它发现了七个有限奇点，恢复了精确根 $0, 9$、十四个生效的共振门禁和最大对数次数一。下一步的发布里程碑是：一个公开的平直时空费曼积分例子、与专门有理系数生成器在相同输入下的基准对比、可安装的命令行接口、重复根形式块解耦、分支、代数扩张以及 Stokes 匹配。凡是局部分支数据不足，程序包将继续保持拒绝。

参考文献

[1] K. G. Chetyrkin and F. V. Tkachov. Integration by parts: The algorithm to calculate β-functions in 4 loops. *Nucl. Phys. B*, 192:159–204, 1981.

[2] T. Gehrmann and E. Remiddi. Differential equations for two-loop four-point functions. *Nucl. Phys. B*, 580:485–518, 2000.

[3] Martijn Hidding. DiffExp, a Mathematica package for computing Feynman integrals in terms of one-dimensional series expansions. *Comput. Phys. Commun.*, 269:108125, 2021.

[4] Xiao Liu and Yan-Qing Ma. AMFlow: A Mathematica package for Feynman integrals computation via auxiliary mass flow. *Comput. Phys. Commun.*, 283:108565, 2023.

[5] Rui-Jun Huang, Xiao Liu, and Yan-Qing Ma. AMFlow 2.0: significant algorithmic and software improvements for Feynman integral evaluation. 7 2026.

[6] Wen Chen. Semi-automatic calculations of multi-loop Feynman amplitudes with AmpRed. *Comput. Phys. Commun.*, 312:109607, 2025.

[7] Roman N. Lee and Andrei A. Pomeransky. Critical points and number of master integrals. *JHEP*, 11:165, 2013.

[8] M. Beneke and Vladimir A. Smirnov. Asymptotic expansion of Feynman integrals near threshold. *Nucl. Phys. B*, 522:321–344, 1998.

[9] A. Pak and A. Smirnov. Geometric approach to asymptotic expansion of Feynman integrals. *Eur. Phys. J. C*, 71:1626, 2011.

[10] Xingang Chen and Yi Wang. Large non-Gaussianities with Intermediate Shapes from Quasi-Single Field Inflation. *Phys. Rev. D*, 81:063511, 2010.

[11] Xingang Chen and Yi Wang. Quasi-Single Field Inflation and Non-Gaussianities. *JCAP*, 04:027, 2010.

[12] Nima Arkani-Hamed and Juan Maldacena. Cosmological Collider Physics. 3 2015.

[13] Jiaqi Chen and Bo Feng. Towards systematic evaluation of de Sitter correlators via Generalized Integration-By-Parts relations. *JHEP*, 06:199, 2024.

[14] Jiaqi Chen, Bo Feng, and Yi-Xiao Tao. Multivariate hypergeometric solutions of cosmological (dS) correlators by d log-form differential equations. *JHEP*, 03:075, 2025.

[15] Jiaqi Chen, Bo Feng, Zhehan Qin, and Yi-Xiao Tao. Loop integrals in de Sitter spacetime: The parity-split IBP system and d log-form differential equations. 4 2026.

[16] Nima Arkani-Hamed, Daniel Baumann, Aaron Hillman, Austin Joyce, Hayden Lee, and Guilherme L. Pimentel. Differential equations for cosmological correlators. *JHEP*, 09:009, 2025.

[17] Nima Arkani-Hamed, Daniel Baumann, Aaron Hillman, Austin Joyce, Hayden Lee, and Guilherme L. Pimentel. Kinematic Flow and the Emergence of Time. *Phys. Rev. Lett.*, 135(3):031602, 2025.

[18] Shounak De and Andrzej Pokraka. Cosmology meets cohomology. *JHEP*, 03:156, 2024.

[19] Daniel Baumann, Austin Joyce, Hayden Lee, and Kamran Salehi Vaziri. Differential Equations for Massive Correlators. 4 2026.

[20] Kostas Glampedakis, Aaron D. Johnson, and Daniel Kennefick. Darboux transformation in black hole perturbation theory. *Phys. Rev. D*, 96(2):024036, 2017.

[21] E. W. Leaver. An Analytic representation for the quasi normal modes of Kerr black holes. *Proc. Roy. Soc. Lond. A*, 402:285–298, 1985.

[22] Emanuele Berti, Vitor Cardoso, and Andrei O. Starinets. Quasinormal modes of black holes and black branes. *Class. Quant. Grav.*, 26:163001, 2009.

[23] Francesco Moriello. Generalised power series expansions for the elliptic planar families of Higgs + jet production at two loops. *JHEP*, 01:150, 2020.